

Robust Verification

This chapter describes special STA analyses such as time borrowing, clock gating and non-sequential timing checks. In addition, advanced STA concepts such as on-chip variations, statistical timing and trade-off between power and timing are also presented.

10.1 On-Chip Variations

In general, the process and environmental parameters may not be uniform across different portions of the die. Due to process variations, identical MOS transistors in different portions of the die may not have similar characteristics. These differences are due to process variations within the die. Note that the process parameter variations across multiple manufactured lots can cover the entire span of process models from *slow* to *fast* (Section

2.10). In this section, we discuss the analysis of the process variations possible on one die (called *local* process variations) which are much smaller than the variations across multiple manufacturing lots (called *global* process variations).

Besides the variations in the process parameters, different portions of the design may also see different power supply voltage and temperature. It is therefore possible that two regions of the same chip are not at identical PVT conditions. These differences can arise due to many factors, including:

- i. IR drop variation along the die area affecting the local power supply.
- ii. Voltage threshold variation of the PMOS or the NMOS device.
- iii. Channel length variation of the PMOS or the NMOS device.
- iv. Temperature variations due to local hot spots.
- v. Interconnect metal etch or thickness variations impacting the interconnect resistance or capacitance.

The PVT variations described above are referred to as **On-Chip Variations (OCV)** and these variations can affect the wire delays and cell delays in different portions of the chip. As discussed above, modeling of OCV is not intended to model the entire span of the PVT variations possible from wafer to wafer but to model the PVT variations that are possible locally within a single die. The OCV effect is typically more pronounced on clock paths as they travel longer distances in a chip. One way to account for the local PVT variations is to incorporate the OCV analysis during STA. The static timing analysis described in previous chapters obtains the timing at a specific timing corner and does not model the variations along the die. Since the clock and data paths can be affected differently by the OCV, the timing verification can model the OCV effect by making the PVT conditions for the launch and capture paths to be slightly different. The STA can include the OCV effect by derating the delays of specific paths, that is, by making those paths faster or slower and then validating the behavior of the design with these variations. The cell delays or wire delays or both can be derated to model the effect of OCV.

We now examine how the OCV derating is done for a setup check. Consider the logic shown in Figure 10-1 where the PVT conditions can vary along the chip. The worst condition for setup check occurs when the launch clock path and the data path have the OCV conditions which result in the largest delays, while the capture clock path has the OCV conditions which result in the smallest delays. Note that the smallest and largest here are due to local PVT variations on a die.

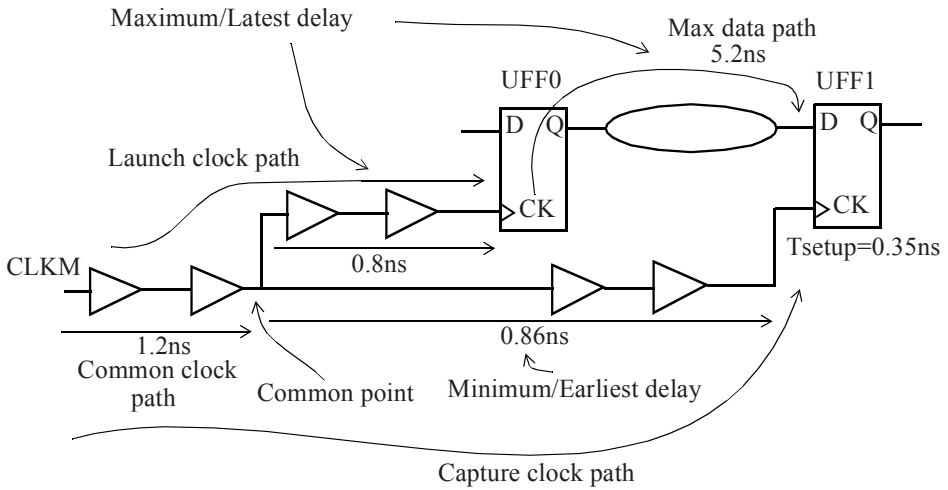


Figure 10-1 Derating setup timing check for OCV.

For this example, here is the setup timing check condition; this does not include any OCV setting for derating delays.

$$\text{LaunchClockPath} + \text{MaxDataPath} \leq \text{ClockPeriod} + \text{CaptureClockPath} - T_{\text{setup_UFF1}}$$

$$\text{This implies that the minimum clock period} = \text{LaunchClockPath} + \text{MaxDataPath} - \text{CaptureClockPath} + T_{\text{setup_UFF1}}$$

From the figure,

LaunchClockPath = $1.2 + 0.8 = 2.0$

MaxDataPath = 5.2

CaptureClockPath = $1.2 + 0.86 = 2.06$

Tsetup_UFF1 = 0.35

This results in a minimum clock period of:

$2.0 + 5.2 - 2.06 + 0.35 = 5.49\text{ns}$

The above path delays correspond to the delay values without any OCV derating. Cell and net delays can be derated using the **set_timing_derate** specification. For example, the commands:

```
set_timing_derate -early 0.8
```

```
set_timing_derate -late 1.1
```

derate the minimum/shortest/early paths by -20% and derate the maximum/longest/latest paths by +10%. Long path delays (for example, data paths and launch clock path for setup checks or capture clock paths for hold checks) are multiplied by the derate value specified using the *-late* option, and short path delays (for example, capture clock paths for setup checks or data paths and launch clock paths for hold checks) are multiplied by the derate values specified using the *-early* option. If no derating factors are specified, a value of 1.0 is assumed.

The derating factors apply uniformly to all net delays and cell delays. If an application scenario warrants different derating factors for cells and nets, the *-cell_delay* and the *-net_delay* options can be used in the *set_timing_derate* specification.

```
# Derate only the cell delays - early paths by -10%, and
```

```
# no derate on the late paths:
```

```
set_timing_derate -cell_delay -early 0.9
```

```
set_timing_derate -cell_delay -late 1.0
```

```
# Derate only the net delays - no derate on the early paths
# and derate the late paths by +20%:
set_timing_derate -net_delay -early 1.0
set_timing_derate -net_delay -late 1.2
```

Cell check delays, such as setup and hold of a cell, can be derated by using the *-cell_check* option. With this option, any output delay specified using *set_output_delay* is also derated as this specification is part of the setup requirement of that output. However, no such implicit derating is applied for the input delays specified using the *set_input_delay* specification.

```
# Derate the cell timing check values:
set_timing_derate -early 0.8 -cell_check
set_timing_derate -late 1.1 -cell_check

# Derate the early clock paths:
set_timing_derate -early 0.95 -clock
# Derate the late data paths:
set_timing_derate -late 1.05 -data
```

The *-clock* option (shown above) applies derating only to clock paths. Similarly, the *-data* option applies derating only to data paths.

We now apply the following derating to the example of Figure 10-1.

```
set_timing_derate -early 0.9
set_timing_derate -late 1.2
set_timing_derate -late 1.1 -cell_check
```

With these derating values, we get the following for setup check:

```
LaunchClockPath = 2.0 * 1.2 = 2.4
MaxDataPath = 5.2 * 1.2 = 6.24
CaptureClockPath = 2.06 * 0.9 = 1.854
Tsetup_UFF1 = 0.35 * 1.1 = 0.385
```

This results in a minimum clock period of:
 $2.4 + 6.24 - 1.854 + 0.385 = 7.171\text{ns}$

In the setup check above, there is a discrepancy since the *common clock path* (Figure 10-1) of the clock tree, with a delay of 1.2ns, is being derated differently for the launch clock and for the capture clock. This part of the clock tree is common to both the launch clock and the capture clock and should not be derated differently. Applying different derating for the launch and capture clock is overly pessimistic as in reality this part of the clock tree will really be at only one PVT condition, either as a maximum path or as a minimum path (or anything in between) but never both at the same time. The pessimism caused by different derating factors applied on the common part of the clock tree is called **Common Path Pessimism** (CPP) which should be removed during the analysis. CPPR, which stands for *Common Path Pessimism Removal*, is often listed as a separate item in a path report. It is also labeled as *Clock Reconvergence Pessimism Removal* (CRPR).

CPPR is the removal of artificially induced pessimism between the launch clock path and the capture clock path in timing analysis. If the same clock drives both the capture and the launch flip-flops, then the clock tree will likely share a common portion before branching. CPP itself is the delay difference along this common portion of the clock tree due to different deratings for launch and capture clock paths. The difference between the minimum and the maximum arrival times of the clock signal at the common point is the CPP. The *common point* is defined as the output pin of the last cell in the common portion of the clock tree.

$$\text{CPP} = \text{LatestArrivalTime@CommonPoint} - \text{EarliestArrivalTime@CommonPoint}$$

The *Latest* and *Earliest* times in the above analysis are in reference to the OCV derating at a specific timing corner - for example worst-case slow or best-case fast. For the example of Figure 10-1,

$$\begin{aligned} \text{LatestArrivalTime@CommonPoint} &= 1.2 * 1.2 = 1.44 \\ \text{EarliestArrivalTime@CommonPoint} &= 1.2 * 0.9 = 1.08 \end{aligned}$$

This implies a CPP of: $1.44 - 1.08 = 0.36\text{ns}$

With the CPP correction, this results in a minimum clock period of: $7.171 - 0.36 = 6.811\text{ns}$

Applying the OCV derating has increased the minimum clock period from 5.49ns to 6.811ns for this example design. This illustrates that the OCV variations modeled by these derating factors can reduce the maximum frequency of operation of the design.

Analysis with OCV at Worst PVT Condition

If the setup timing check is being performed at the worst-case PVT condition, no derating is necessary on the late paths as they are already the worst possible. However, derating can be applied to the early paths by making those paths faster by using a specific derating, for example, speeding up the early paths by 10%. A derate specification at the worst-case slow corner may be something like:

```
set_timing_derate -early 0.9
set_timing_derate -late 1.0
# Don't derate the late paths as they are already the slowest,
# but derate the early paths to make these faster by 10%.
```

The above derate settings are for max path (or setup) checks at the worst-case slow corner; thus the late path OCV derate setting is kept at 1.0 so as not to slow it beyond the worst-case slow corner.

An example of setup timing check at the worst-case slow corner is described next. The derate specification is specified for the capture clock path below:

```
# Derate the early clock paths:
set_timing_derate -early 0.8 -clock
```

Here is the setup timing check path report performed at the worst-case slow corner. The derating used by the late paths are reported as *Max Data*

Paths Derating Factor and as *Max Clock Paths Derating Factor*. The derating used for the early paths is reported as *Min Clock Paths Derating Factor*.

```
Startpoint: UFF0 (rising edge-triggered flip-flop clocked by CLKM)
Endpoint: UFF1 (rising edge-triggered flip-flop clocked by CLKM)
Path Group: CLKM
Path Type: max
Max Data Paths Derating Factor : 1.000
Min Clock Paths Derating Factor : 0.800
Max Clock Paths Derating Factor : 1.000
```

Point	Incr	Path

clock CLKM (rise edge)	0.000	0.000
clock source latency	0.000	0.000
CLKM (in)	0.000	0.000 r
UCKBUF0/C (CKB)	0.056	0.056 r
UCKBUF1/C (CKB)	0.058	0.114 r
UFF0/CK (DF)	0.000	0.114 r
UFF0/Q (DF) <=	0.143	0.258 f
UNOR0/ZN (NR2)	0.043	0.301 r
UBUF4/Z (BUFF)	0.052	0.352 r
UFF1/D (DF)	0.000	0.352 r
data arrival time		0.352
clock CLKM (rise edge)	10.000	10.000
clock source latency	0.000	10.000
CLKM (in)	0.000	10.000 r
UCKBUF0/C (CKB)	0.045	10.045 r
UCKBUF2/C (CKB)	0.054	10.098 r
UFF1/CK (DF)	0.000	10.098 r
clock reconvergence pessimism	0.011	10.110
clock uncertainty	-0.300	9.810
library setup time	-0.044	9.765
data required time		9.765

data required time		9.765
data arrival time		-0.352

slack (MET)		9.413

Notice that the capture clock path is derated by 20%. See cell *UCKBUF0* in the timing report. In the launch path, it has a delay of 56ps, while it has a derated delay of 45ps in the capture path. The cell *UCKBUF0* is on the common clock path, that is, on both the capture clock path and the launch clock path. Since the common clock path cannot have a different derating, the difference in timing for this common path, 56ps - 45ps = 11ps, is corrected separately. This appears as the line *clock reconvergence pessimism* in the report. In summary, if one were to compare the reports of this path, with and without derating, one would notice that only the cell and net delays for the capture clock path have been derated.

OCV for Hold Checks

We now examine how the derating is done for a hold timing check. Consider the logic shown in Figure 10-2. If the PVT conditions are different along the chip, the worst condition for hold check occurs when the launch clock path and the data path have OCV conditions which result in the smallest delays, that is, when we have the earliest launch clock, and the capture clock path has the OCV conditions which result in the largest delays, that is, has the latest capture clock.

The hold timing check is specified in the following expression for this example.

```
LaunchClockPath + MinDataPath - CaptureClockPath -
  Thold_UFF1 >= 0
```

Applying the delay values in the Figure 10-2 to the expression, we get (without applying any derating):

```
LaunchClockPath = 0.25 + 0.6 = 0.85
MinDataPath = 1.7
CaptureClockPath = 0.25 + 0.75 = 1.00
Thold_UFF1 = 1.25
```

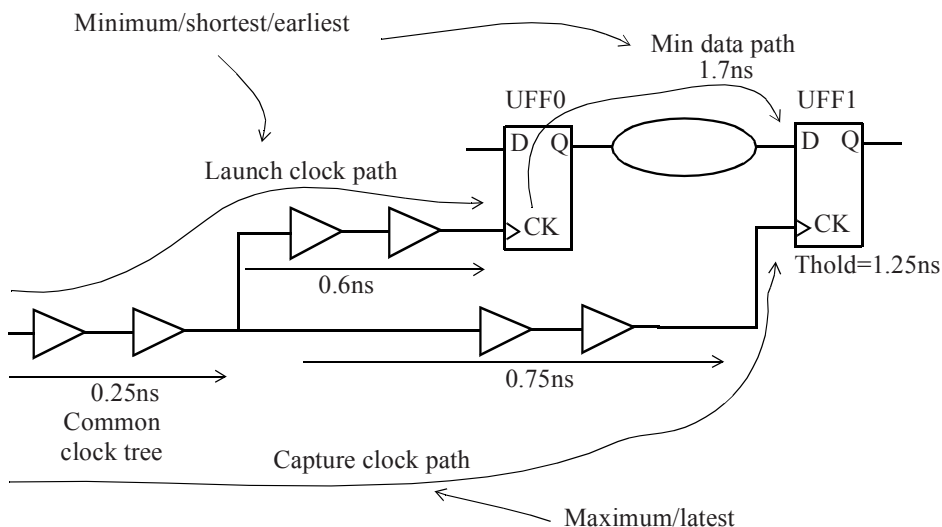


Figure 10-2 Derating hold timing check for OCV.

This implies that the condition is:

$$0.85 + 1.7 - 1.00 - 1.25 = 0.3\text{n} \geq 0$$

which is true, and thus no hold violation exists.

Applying the following derate specification:

```
set_timing_derate -early 0.9
set_timing_derate -late 1.2
set_timing_derate -early 0.95 -cell_check
```

we get:

$$\begin{aligned}\text{LaunchClockPath} &= 0.85 * 0.9 = 0.765 \\ \text{MinDataPath} &= 1.7 * 0.9 = 1.53\end{aligned}$$

```
CaptureClockPath = 1.00 * 1.2 = 1.2
Thold_UFF1 = 1.25 * 0.95 = 1.1875
```

Common clock path pessimism: $0.25 * (1.2 - 0.9) = 0.075$

Common clock path pessimism created by applying derating on the common clock tree for both launch and capture clock paths is also removed for hold timing checks. The hold check condition then becomes:

$$0.765 + 1.53 - 1.2 - 1.1875 + 0.075 = -0.0175\text{ns}$$

which is less than 0, thus showing that there is a hold violation with the OCV derating factors applied to the early and late paths.

In general, the hold timing check is performed at the best-case fast PVT corner. In such a scenario, no derating is necessary on the early paths, as those paths are already the earliest possible. However, derating can be applied on the late paths by making these slower by a specific derating factor, for example, slowing the late paths by 20%. A derate specification at this corner would be something like:

```
set_timing_derate -early 1.0
set_timing_derate -late 1.2
# Don't derate the early paths as they are already the
# fastest, but derate the late paths slower by 20%.
```

In the example of Figure 10-2,

```
LatestArrivalTime@CommonPoint = 0.25 * 1.2 = 0.30
EarliestArrivalTime@CommonPoint = 0.25 * 1.0 = 0.25
```

This implies a common path pessimism of:

$$0.30 - 0.25 = 0.05\text{ns}$$

Here is a hold timing check path report for an example design that uses this derating.

Startpoint: UFF0 (rising edge-triggered flip-flop clocked by CLKM)
Endpoint: UFF1 (rising edge-triggered flip-flop clocked by CLKM)
Path Group: CLKM
Path Type: min
Min Data Paths Derating Factor : 1.000
Min Clock Paths Derating Factor : 1.000
Max Clock Paths Derating Factor : 1.200

Point	Incr	Path

clock CLKM (rise edge)	0.000	0.000
clock source latency	0.000	0.000
CLKM (in)	0.000	0.000 r
UCKBUF0/C (CKB)	0.056	0.056 r
UCKBUF1/C (CKB)	0.058	0.114 r
UFF0/CK (DF)	0.000	0.114 r
UFF0/Q (DF) <=	0.144	0.258 r
UNOR0/ZN (NR2)	0.021	0.279 f
UBUF4/Z (BUFF)	0.055	0.334 f
UFF1/D (DF)	0.000	0.334 f
data arrival time		0.334
clock CLKM (rise edge)	0.000	0.000
clock source latency	0.000	0.000
CLKM (in)	0.000	0.000 r
UCKBUF0/C (CKB)	0.067	0.067 r
UCKBUF2/C (CKB)	0.080	0.148 r
UFF1/CK (DF)	0.000	0.148 r
clock reconvergence pessimism	-0.011	0.136
clock uncertainty	0.050	0.186
library hold time	0.015	0.201
data required time		0.201

data required time		0.201
data arrival time		-0.334

slack (MET)		0.133

Notice that the late paths are derated by +20% while the early paths are not derated. See cell *UCKBUF0*. Its delay on the launch path is 56ps while the delay on the capture path is 67ps - derated by +20%. *UCKBUF0* is the cell on the common clock tree and thus the pessimism introduced due to different derating on this common clock tree is, $67\text{ps} - 56\text{ps} = 11\text{ps}$, which is accounted for separately on the line *clock reconvergence pessimism*.

10.2 Time Borrowing

The time borrowing technique, which is also called **cycle stealing**, occurs at a latch. In a latch, one edge of the clock makes the latch transparent, that is, it opens the latch so that output of the latch is the same as the data input; this clock edge is called the **opening edge**. The second edge of the clock closes the latch, that is, any change on the data input is no longer available at the output of the latch; this clock edge is called the **closing edge**.

Typically, the data should be ready at a latch input before the active edge of the clock. However, since a latch is transparent when the clock is active, the data can arrive later than the active clock edge, that is, it can borrow time from the next cycle. If such time is borrowed, the time available for the following stage (latch to another sequential cell) is reduced.

Figure 10-3 shows an example of time borrowing using an active rising edge. If data *DIN* is ready at time *A* prior to the latch opening on the rising edge of *CLK* at 10ns, the data flows to the output of the latch as it opens. If data arrives at time *B* as shown for *DIN (delayed)*, it borrows time *T_b*. However, this reduces the time available from the latch to the next flip-flop *UFF2* - instead of a complete clock cycle, only time *T_a* is available.

The first rule in timing to a latch is that if the data arrives before the opening edge of the latch, the behavior is modeled exactly like a flip-flop. The opening edge captures the data and the same clock edge launches the data as the start point for the next path.

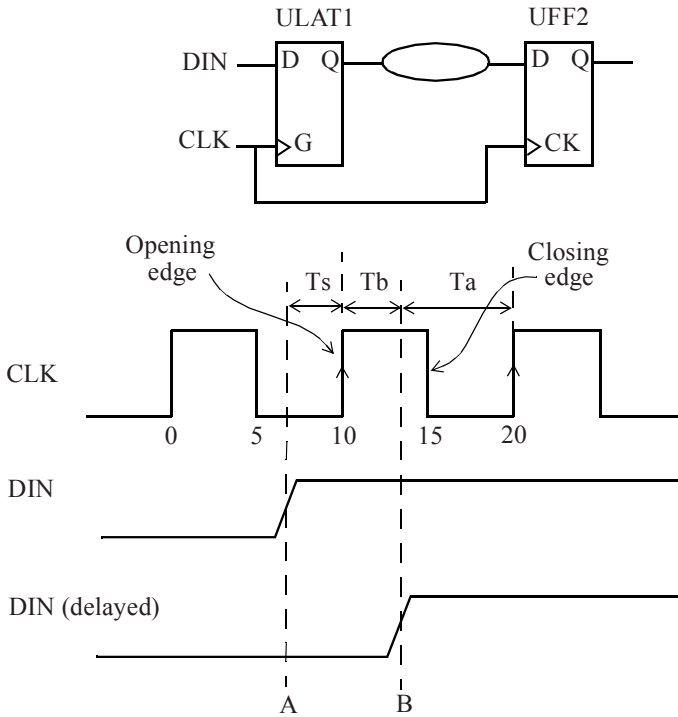


Figure 10-3 *Time borrowing.*

The second rule applies when the data signal arrives while the latch is transparent (between the opening and the closing edge). The output of the latch, rather than the clock pin, is used as the launch point for the next stage. The amount of time borrowed by the path ending at the latch determines the launch time for the next stage.

A data signal that arrives after the closing edge at the latch is a timing violation. Figure 10-4 shows the timing regions for data arrival for positive slack, zero slack, and negative slack (that is, when a violation occurs).

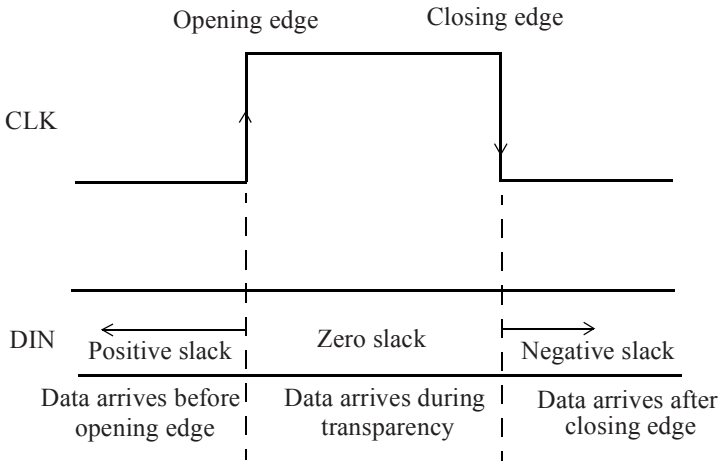


Figure 10-4 *Latch timing violation windows.*

Figure 10-5(a) shows the use of a latch with a half-cycle path to the next stage flip-flop. Figure 10-5(b) depicts the waveforms for the scenario of time borrowing. The clock period is 10ns. The data is launched by *UFF0* at time 0, but the data path takes 7ns. The latch *ULAT1* opens up at 5ns. Thus 2ns is borrowed from the path *ULAT1* to *UFF1*. The time available for the *ULAT1* to *UFF1* path is only 3ns (5ns - 2ns).

We next describe three sets of timing reports for the latch example in Figure 10-5(a) to illustrate the different amounts of time borrowed from the next stage.

Example with No Time Borrowed

Here is the setup path report when the data path delay from the flip-flop *UFF0* to the latch *ULAT1* is less than 5ns.

```
Startpoint: UFF0 (rising edge-triggered flip-flop clocked by CLK)
Endpoint: ULAT1 (positive level-sensitive latch clocked by CLK')
Path Group: CLK
```

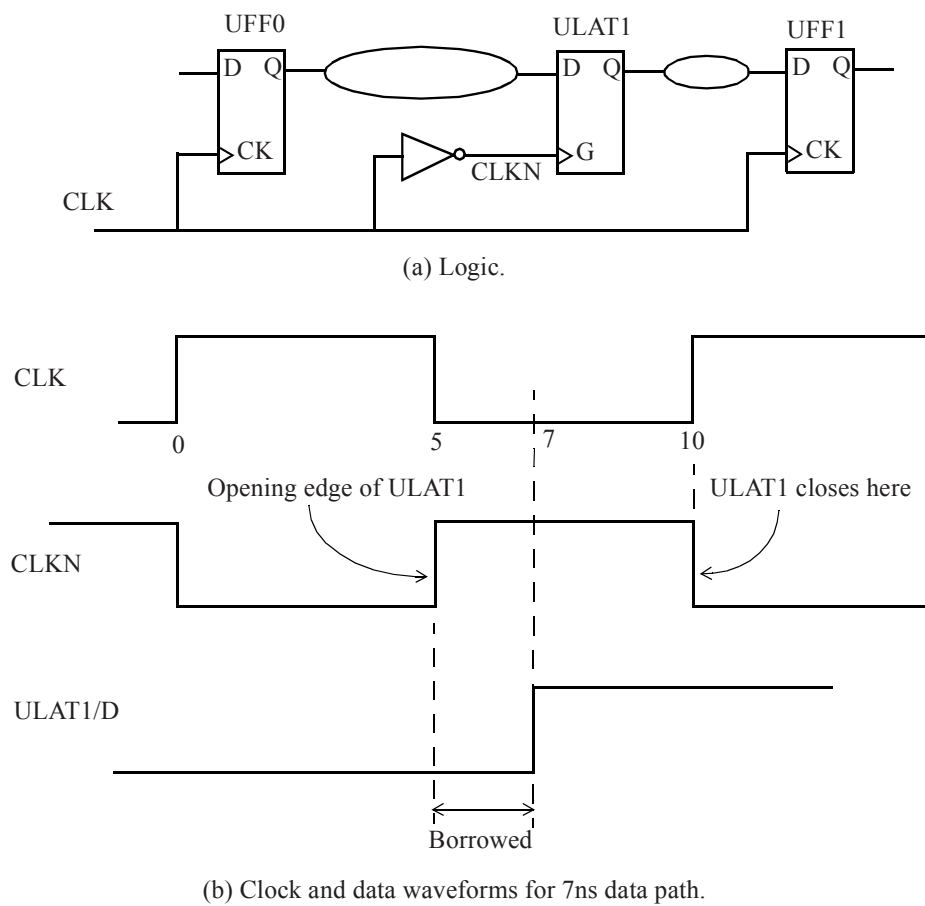


Figure 10-5 *Time borrowing example.*

Path Type: max

Point	Incr	Path
clock CLK (rise edge)	0.00	0.00
clock source latency	0.00	0.00

clk (in)	0.00	0.00 r
UFF0/CK (DF)	0.00	0.00 r
UFF0/Q (DF)	0.12	0.12 r
UBUF0/Z (BUFF)	2.01	2.13 r
UBUF1/Z (BUFF)	2.46	4.59 r
UBUF2/Z (BUFF)	0.07	4.65 r
ULAT1/D (LH)	0.00	4.65 r
data arrival time		4.65
clock CLK' (rise edge)	5.00	5.00
clock source latency	0.00	5.00
clk (in)	0.00	5.00 f
UINV0/ZN (INV)	0.02	5.02 r
ULAT1/G (LH)	0.00	5.02 r
time borrowed from endpoint	0.00	5.02
data required time		5.02

data required time		5.02
data arrival time		-4.65

slack (MET)		0.36
Time Borrowing Information		

CLK' nominal pulse width	5.00	
clock latency difference	-0.00	
library setup time	-0.01	

max time borrow	4.99	
actual time borrow	0.00	

In this case, there is no need to borrow as data reaches the latch *ULAT1* in time before the latch opens.

Example with Time Borrowed

The path report below shows the case where the data path delay from the flip-flop *UFF0* to the latch *ULAT1* is greater than 5ns.

Startpoint: UFF0 (rising edge-triggered flip-flop clocked by CLK)
Endpoint: ULAT1 (positive level-sensitive latch clocked by CLK')
Path Group: CLK
Path Type: max

Point	Incr	Path

clock CLK (rise edge)	0.00	0.00
clock source latency	0.00	0.00
clk (in)	0.00	0.00 r
UFF0/CK (DF)	0.00	0.00 r
UFF0/Q (DF)	0.12	0.12 r
UBUF0/Z (BUFF)	3.50	3.62 r
UBUF1/Z (BUFF)	3.14	6.76 r
UBUF2/Z (BUFF)	0.07	6.83 r
ULAT1/D (LH)	0.00	6.83 r
data arrival time		6.83
clock CLK' (rise edge)	5.00	5.00
clock source latency	0.00	5.00
clk (in)	0.00	5.00 f
UINV0/ZN (INV)	0.02	5.02 r
ULAT1/G (LH)	0.00	5.02 r
time borrowed from endpoint	1.81	6.83
data required time		6.83

data required time		6.83
data arrival time		-6.83

slack (MET)		0.00
Time Borrowing Information		

CLK' nominal pulse width	5.00	
clock latency difference	-0.00	
library setup time	-0.01	

max time borrow	4.99
actual time borrow	1.81

In this case, since the data becomes available while the latch is transparent, the required delay of 1.81ns is borrowed from the subsequent path and the timing is still met. Here is the path report of the subsequent path showing that 1.81ns was already borrowed by the previous path.

Startpoint: ULAT1 (positive level-sensitive latch clocked by CLK')
 Endpoint: UFF1 (rising edge-triggered flip-flop clocked by CLK)
 Path Group: CLK
 Path Type: max

Point	Incr	Path

clock CLK' (rise edge)	5.00	5.00
clock source latency	0.00	5.00
clk (in)	0.00	5.00 f
UINV0/ZN (INV)	0.02	5.02 r
ULAT1/G (LH)	0.00	5.02 r
time given to startpoint	1.81	6.83
ULAT1/QN (LH)	0.13	6.95 f
UFF1/D (DF)	0.00	6.95 f
data arrival time		6.95
clock CLK (rise edge)	10.00	10.00
clock source latency	0.00	10.00
clk (in)	0.00	10.00 r
UFF1/CK (DF)	0.00	10.00 r
library setup time	-0.04	9.96
data required time		9.96

data required time		9.96
data arrival time		-6.95

slack (MET)		3.01

Example with Timing Violation

In this case, the data path delay is much larger and data becomes available only after the latch closes. This is clearly a timing violation.

Startpoint: UFF0 (rising edge-triggered flip-flop clocked by CLK)
Endpoint: ULAT1 (positive level-sensitive latch clocked by CLK')
Path Group: CLK
Path Type: max

Point	Incr	Path

clock CLK (rise edge)	0.00	0.00
clock source latency	0.00	0.00
clk (in)	0.00	0.00 r
UFF0/CK (DF)	0.00	0.00 r
UFF0/Q (DF)	0.12	0.12 r
UBUF0/Z (BUFF)	6.65	6.77 r
UBUF1/Z (BUFF)	4.33	11.10 r
UBUF2/Z (BUFF)	0.07	11.17 r
ULAT1/D (LH)	0.00	11.17 r
data arrival time		11.17
clock CLK' (rise edge)	5.00	5.00
clock source latency	0.00	5.00
clk (in)	0.00	5.00 f
UINV0/ZN (INV)	0.02	5.02 r
ULAT1/G (LH)	0.00	5.02 r
time borrowed from endpoint	4.99	10.00
data required time		10.00

data required time		10.00
data arrival time		-11.17

slack (VIOLATED)		-1.16
Time Borrowing Information		

CLK' nominal pulse width	5.00	
clock latency difference	-0.00	
library setup time	-0.01	

max time borrow	4.99
actual time borrow	4.99

10.3 Data to Data Checks

Setup and hold checks can also be applied between any two arbitrary data pins, neither of which is a clock. One pin is the **constrained pin**, which acts like a data pin of a flip-flop, and the second pin is the **related pin**, which acts like a clock pin of a flip-flop. One important distinction with respect to the setup check of a flip-flop is that the data to data setup check is performed on the same edge as the launch edge (unlike a normal setup check of a flip-flop, where the capture clock edge is normally one cycle away from the launch clock edge). Thus, the data to data setup checks are also referred to as **zero-cycle checks** or **same-cycle checks**.

A data to data check is specified using the **set_data_check** constraint. Here are example SDC specifications.

```
set_data_check -from SDA -to SCTRL -setup 2.1
set_data_check -from SDA -to SCTRL -hold 1.5
```

See Figure 10-6. *SDA* is the related pin and *SCTRL* is the constrained pin. The setup data check implies that *SCTRL* should arrive at least 2.1ns prior to the edge of the related pin *SDA*. Otherwise it is a data to data setup check violation. The hold data check specifies that *SCTRL* should arrive at least 1.5ns after *SDA*. If the constrained signal arrives earlier than this specification, then it is a data to data hold check violation.

This check is useful in a custom-designed block where it may be necessary to provide specific arrival times of one signal with respect to another. One such common situation is that of a data signal gated by an enable signal and it is required to ensure that the enable signal is stable when the data signal arrives.

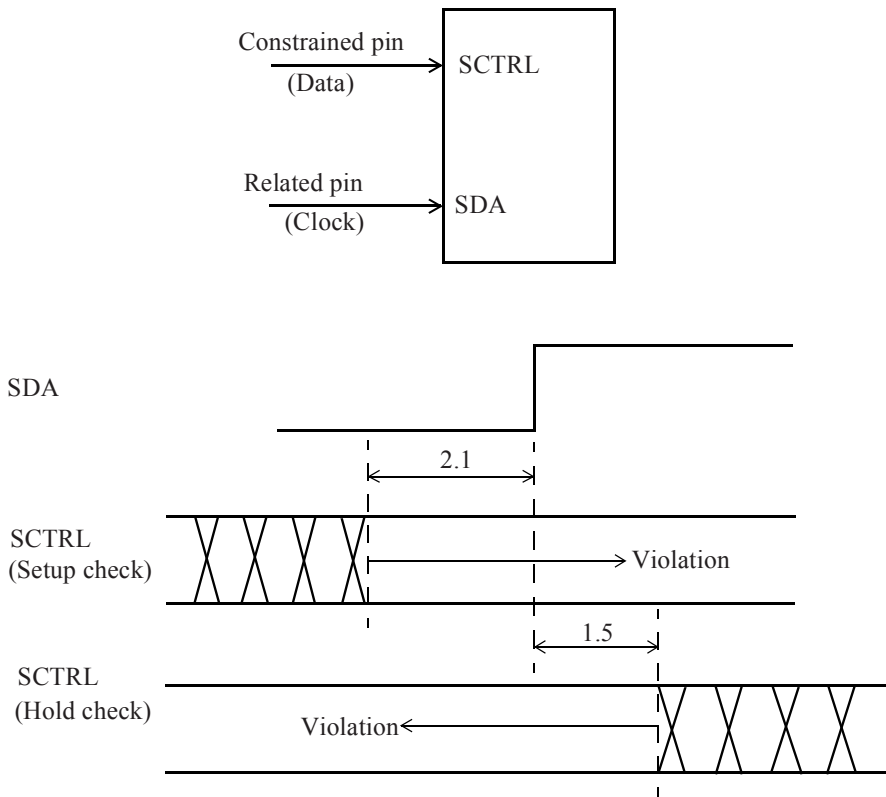


Figure 10-6 *Data to data checks.*

Consider the *and* cell shown in Figure 10-7. We assume the requirement is to ensure that *PNA* arrives 1.8ns before the rising edge of *PREAD* and that it should not change for 1.0ns after the rising edge of *PREAD*. In this example, *PNA* is the constrained pin and *PREAD* is the related pin. The required waveforms are shown in Figure 10-7.

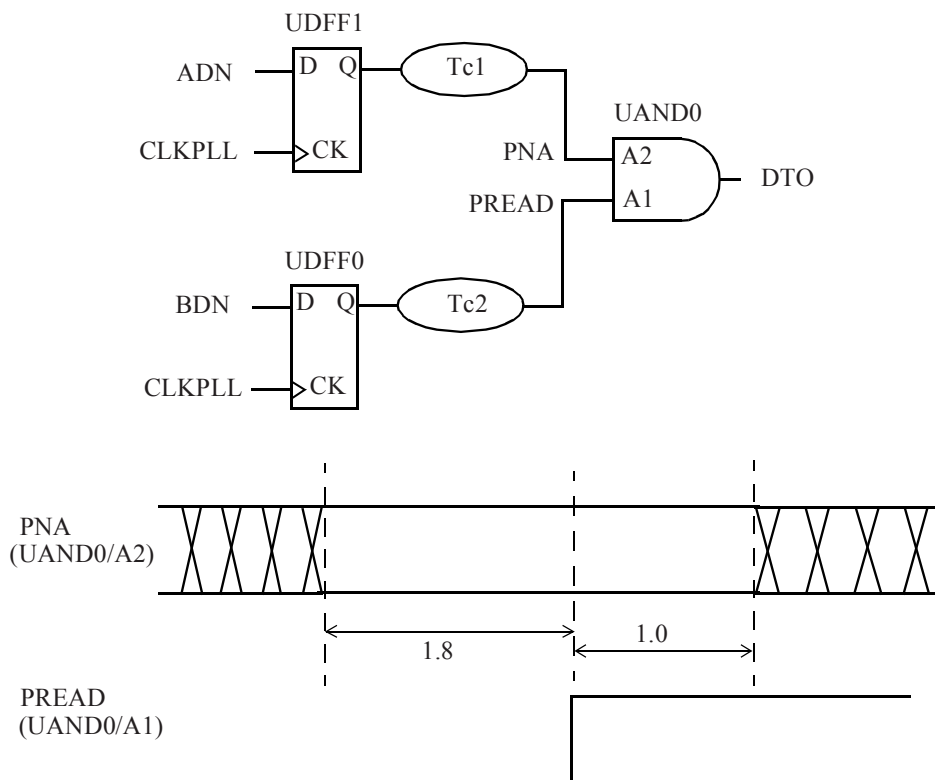


Figure 10-7 Setup and hold timing checks between PNA and PREAD.

Such a requirement can be specified using a data to data setup and hold check.

```
set_data_check -from UAND0/A1 -to UAND0/A2 -setup 1.8
set_data_check -from UAND0/A1 -to UAND0/A2 -hold 1.0
```

Here is the setup report.

Startpoint: UDF1
 (rising edge-triggered flip-flop clocked by CLKPLL)
Endpoint: UAND0
 (rising edge-triggered data to data check clocked by CLKPLL)
Path Group: CLKPLL
Path Type: max

Point	Incr	Path

clock CLKPLL (rise edge)	0.00	0.00
clock source latency	0.00	0.00
CLKPLL (in)	0.00	0.00 r
UDF1/CK (DF)	0.00	0.00 r
UDF1/Q (DF)	0.12	0.12 f
UBUF0/Z (BUFF)	0.06	0.18 f
UAND0/A2 (AN2)	0.00	0.18 f
data arrival time		0.18
clock CLKPLL (rise edge)	0.00	0.00
clock source latency	0.00	0.00
CLKPLL (in)	0.00	0.00 r
UDF0/CK (DF)	0.00	0.00 r
UDF0/Q (DF)	0.12	0.12 r
UBUF1/Z (BUFF)	0.05	0.17 r
UBUF2/Z (BUFF)	0.05	0.21 r
UBUF3/Z (BUFF)	0.05	0.26 r
UAND0/A1 (AN2)	0.00	0.26 r
data check setup time	-1.80	-1.54
data required time		-1.54

data required time		-1.54
data arrival time		-0.18

slack (VIOLATED)		-1.72

The setup time is specified as *data check setup time* in the report. The failing report indicates that the *PREAD* needs to be delayed by at least 1.72ns to ensure that *PENA* arrives 1.8ns before *PREAD* - which is our requirement.

One important aspect of a data to data setup check is that the clock edges that launch both the constrained pin and the related pin are from the same clock cycle (also referred to as *same-cycle checks*). Thus notice in the report that the starting time for the capture edge (*UDFF0/CK*) is at 0ns, not one cycle later as one would typically see in a setup report.

The zero-cycle setup check causes the hold timing check to be different from other hold check reports - the hold check is no longer on the same clock edge. Here is the clock specification for *CLKPLL* which is utilized for the hold path report below.

```
create_clock -name CLKPLL -period 10 -waveform {0 5} \  
[get_ports CLKPLL]
```

Startpoint: UDFF1

(rising edge-triggered flip-flop clocked by CLKPLL)

Endpoint: UAND0

(falling edge-triggered data to data check clocked by CLKPLL)

Path Group: CLKPLL

Path Type: min

Point	Incr	Path

clock CLKPLL (rise edge)	10.00	10.00
clock source latency	0.00	10.00
CLKPLL (in)	0.00	10.00 r
UDFF1/CK (DF)	0.00	10.00 r
UDFF1/Q (DF) <-	0.12	10.12 r
UBUF0/Z (BUFF)	0.05	10.17 r
UAND0/A2 (AN2)	0.00	10.17 r
data arrival time		10.17
clock CLKPLL (rise edge)	0.00	0.00
clock source latency	0.00	0.00
CLKPLL (in)	0.00	0.00 r
UDFF0/CK (DF)	0.00	0.00 r
UDFF0/Q (DF)	0.12	0.12 f
UBUF1/Z (BUFF)	0.06	0.18 f

UBUF2/Z (BUFF)	0.05	0.23 f
UBUF3/Z (BUFF)	0.06	0.29 f
UAND0/A1 (AN2)	0.00	0.29 f
data check hold time	1.00	1.29
data required time		1.29

data required time		1.29
data arrival time		-10.17

slack (MET)		8.88

Notice that the clock edge used to launch the related pin for the hold check is one cycle prior to the launch edge for the constrained pin. This is because by definition a hold check is normally performed one cycle prior to the setup capture edge. Since the clock edges for the constrained pin and the related pin are the same for a data to data setup check, the hold check is done one cycle prior to the launch edge.

In some scenarios, a designer may require the data to data hold check to be performed on the same clock cycle. The same cycle hold requirement implies that the clock edge used for the related pin be moved back to where the clock edge for the constrained pin is. This can be achieved by specifying a multicycle of -1.

```
set_multicycle_path -1 -hold -to UAND0/A2
```

Here is the hold timing report for the example above with this multicycle specification.

```
Startpoint: UDF1
(rising edge-triggered flip-flop clocked by CLKPLL)
Endpoint: UAND0
(falling edge-triggered data to data check clocked by CLKPLL)
Path Group: CLKPLL
Path Type: min

Point                               Incr      Path
-----
```

clock CLKPLL (rise edge)	0.00	0.00
clock source latency	0.00	0.00
CLKPLL (in)	0.00	0.00 r
UDFF1/CK (DF)	0.00	0.00 r
UDFF1/Q (DF) <-	0.12	0.12 r
UBUF0/Z (BUFF)	0.05	0.17 r
UAND0/A2 (AN2)	0.00	0.17 r
data arrival time		0.17
clock CLKPLL (rise edge)	0.00	0.00
clock source latency	0.00	0.00
CLKPLL (in)	0.00	0.00 r
UDFF0/CK (DF)	0.00	0.00 r
UDFF0/Q (DF)	0.12	0.12 f
UBUF1/Z (BUFF)	0.06	0.18 f
UBUF2/Z (BUFF)	0.05	0.23 f
UBUF3/Z (BUFF)	0.06	0.29 f
UAND0/A1 (AN2)	0.00	0.29 f
data check hold time	1.00	1.29
data required time		1.29

data required time		1.29
data arrival time		-0.17

slack (VIOLATED)		-1.12

The hold check is now performed using the same clock edge for the constrained pin and the related pin. An alternate way of having the data to data hold check performed in the same cycle is to specify this as a data to data setup check between the pins in the reverse direction.

```
set_data_check -from UAND0/A2 -to UAND0/A1 -setup 1.0
```

The data to data check is also useful in defining a **no-change data check**. This is done by specifying a setup check on the rising edge and a hold check on the falling edge, such that a no-change window gets effectively defined. This is shown in Figure 10-8.

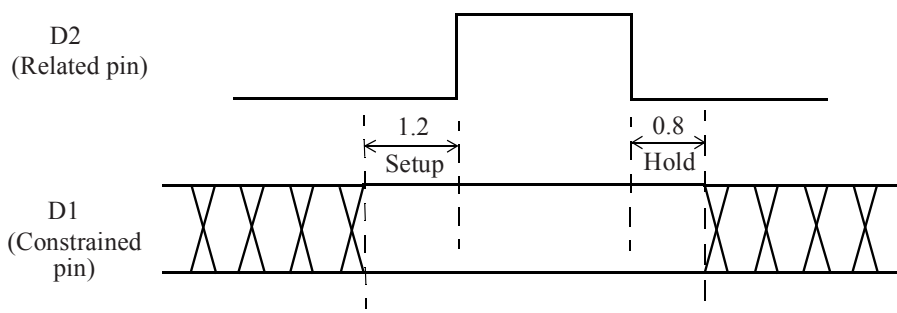


Figure 10-8 *A no-change data check achieved using setup and hold data checks.*

Here are the specifications for this scenario.

```
set_data_check -rise_from D2 -to D1 -setup 1.2
set_data_check -fall_from D2 -to D1 -hold 0.8
```

10.4 Non-Sequential Checks

A library file for a cell or a macro may specify a timing arc to be a non-sequential check, such as a timing arc between two data pins. A non-sequential check is a check between two pins, neither of which is a clock. One pin is the constrained pin that acts like data, while the second pin is the related pin and this acts like a clock. The check specifies how long the data on the constrained pin must be stable before and after the change on the related pin.

Note that this check is specified as part of the cell library specification and no explicit data to data check constraint is required. Here is how such a timing arc may appear in a cell library.

```
pin (WEN) {
  timing () {
    timing_type: non_seq_setup_rising;
```

```

    intrinsic_rise: 1.1;
    intrinsic_fall: 1.15;
    related_pin: "D0";
}
timing () {
    timing_type: non_seq_hold_rising;
    intrinsic_rise: 0.6;
    intrinsic_fall: 0.65;
    related_pin: "D0";
}
}

```

The *setup_rising* refers to the rising edge of the related pin. The intrinsic rise and fall values refer to the rise and fall setup times for the constrained pin. Similar timing arcs can be defined for *hold_rising*, *setup_falling* and *hold_falling*.

A non-sequential check is similar to a data to data check described in Section 10.3, though there are two main differences. In a non-sequential check, the setup and hold values are obtained from the standard cell library, where the setup and hold timing models can be described using a NLDM table model or other advanced timing models. In a data to data check, only a single value can be specified for the data to data setup or hold check. The second difference is that a non-sequential check can only be applied to pins of a cell, whereas a data to data check can be applied to any two arbitrary pins in a design.

A **non-sequential setup check** specifies how early the constrained signal must arrive relative to the related pin. This is shown in Figure 10-9. The cell library contains the setup arc *D0->WEN* which is specified as a non-sequential arc. If the *WEN* signal occurs within the setup window, the non-sequential setup check fails.

A **non-sequential hold check** specifies how late the constrained signal must arrive relative to the related pin. See Figure 10-9. If *WEN* changes within the hold window, then the hold check fails.

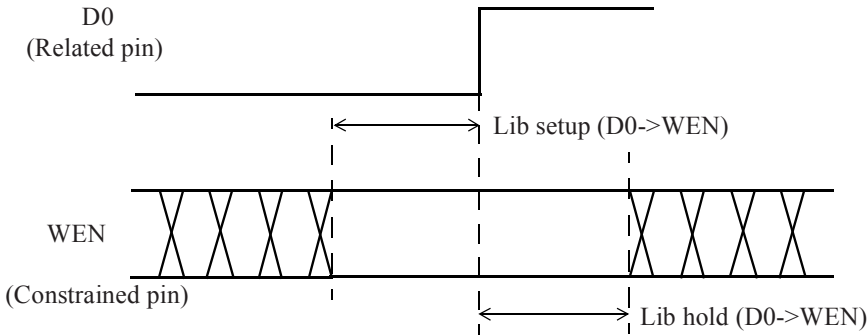


Figure 10-9 *A non-sequential setup and hold check.*

10.5 Clock Gating Checks

A clock gating check occurs when a *gating signal* can control the path of a *clock signal* at a logic cell. An example is shown in Figure 10-10. The pin of the logic cell connected to the clock is called the **clock pin** and the pin where the gating signal is connected to is the **gating pin**. The logic cell where the clock gating occurs is also referred to as the **gating cell**.

One condition for a clock gating check is that the clock that goes through the cell must be used as a clock downstream. The downstream clock usage can be either as a flip-flop clock or it can fanout to an output port or as a generated clock that refers to the output of the gating cell as its master. If the clock is not used as a clock after the gating cell, then no clock gating check is inferred.

Another condition for the clock gating check applies to the gating signal. The signal at the gating pin of the check should not be a clock or if it is a clock, it should not be used as a clock downstream (an example of a clock as a gating signal is included later in this section).

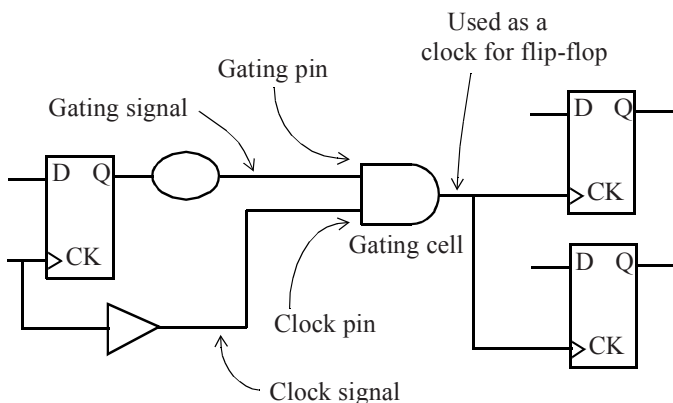


Figure 10-10 A clock gating check.

In a general scenario, the clock signal and the gating signal do not need to be connected to a single logic cell such as *and*, or *or*, but may be inputs to an arbitrary logic block. In such cases, for a clock gating check to be inferred, the clock pin of the check and the gating pin of the check must fan out to a common output pin.

There are two types of clock gating checks inferred:

- *Active-high clock gating check*: Occurs when the gating cell has an *and* or a *nand* function.
- *Active-low clock gating check*: Occurs when the gating cell has an *or* or a *nor* function.

The active-high and active-low refer to the logic state of the gating signal which activates the clock signal at the output of the gating cell. If the gating cell is a complex function where the gating relationship is not obvious, such as a multiplexer or an *xor* cell, STA output will typically provide a warning that no clock gating check is being inferred. However this can be changed by specifying a clock gating relationship for the gating cell explicitly by using the command `set_clock_gating_check`. In such cases, if the

`set_clock_gating_check` specification disagrees with the functionality of the gating cell, the STA will normally provide a warning. We present examples of such commands later in this section.

As specified earlier, a clock can be a gating signal only if it is not used as a clock downstream. Consider the example in Figure 10-11. *CLKB* is not used as a clock downstream due to the definition of the generated clock of *CLKA* - the path of *CLKB* is blocked by the generated clock definition. Hence a clock gating check for clock *CLKA* is inferred for the *and* cell.

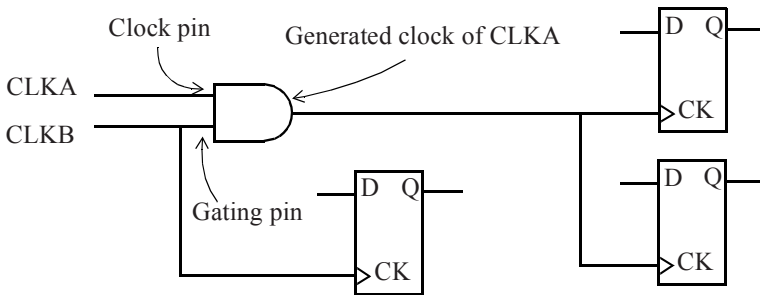


Figure 10-11 *Gating check inferred - clock at the gating pin not used as a clock downstream.*

Active-High Clock Gating

We now examine the timing relationship of an active-high clock gating check. This occurs at an *and* or a *nand* cell; an example using *and* is shown in Figure 10-12. Pin *B* of the gating cell is the clock signal, and pin *A* of the gating cell is the gating signal.

Let us assume that both clocks *CLKA* and *CLKB* have the same waveforms.

```
create_clock -name CLKA -period 10 -waveform {0 5} \
[get_ports CLKA]
create_clock -name CLKB -period 10 -waveform {0 5} \
[get_ports CLKB]
```

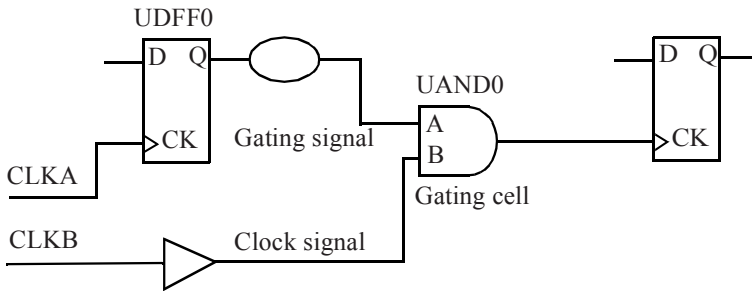



Figure 10-12 *Active high clock gating using an AND cell.*

Because it is an *and* cell, a high on gating signal *UAND0/A* opens up the gating cell and allows the clock to propagate through. The clock gating check is intended to validate that the gating pin transition does not create an active edge for the fanout clock. For positive edge-triggered logic, this implies that the rising edge of the gating signal occurs during the inactive period of the clock (when it is low). Similarly, for negative edge-triggered logic, the falling edge of the gating signal should occur only when the clock is low. Note that if the clock drives both positive and negative edge-triggered flip-flops, any transition of the gating signal (rising or falling edge) must occur only when the clock is low. Figure 10-13 shows an example of a gating signal transition during the active edge which needs to be delayed to pass the clock gating check.

The active-high clock gating setup check requires that the gating signal changes before the clock goes high. Here is the setup path report.

```

Startpoint: UDF0
  (rising edge-triggered flip-flop clocked by CLKA)
Endpoint: UAND0
  (rising clock gating-check end-point clocked by CLKB)
Path Group: **clock_gating_default**
Path Type: max

```

Point	Incr	Path

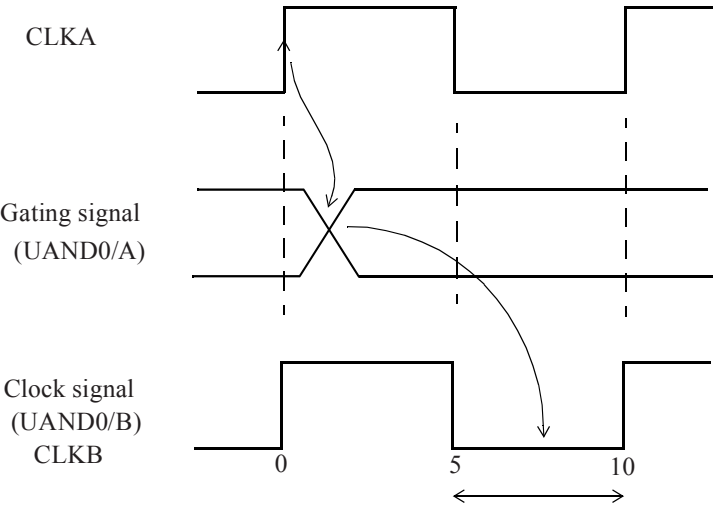


Figure 10-13 Gating signal needs to be delayed.

clock CLK A (rise edge)	0.00	0.00
clock source latency	0.00	0.00
CLK A (in)	0.00	0.00 r
UDDF0/CK (DF)	0.00	0.00 r
UDDF0/Q (DF)	0.13	0.13 f
UAND0/A1 (AN2)	0.00	0.13 f
data arrival time		0.13
clock CLK B (rise edge)	10.00	10.00
clock source latency	0.00	10.00
CLK B (in)	0.00	10.00 r
UAND0/A2 (AN2)	0.00	10.00 r
clock gating setup time	0.00	10.00
data required time		10.00

data required time		10.00
data arrival time		-0.13

slack (MET)		9.87

Notice that the *Endpoint* indicates that it is a clock gating check. In addition, the path is in the *clock_gating_default* group of paths as specified in *Path Group*. The check validates that the gating signal changes before the next rising edge of clock *CLKB* at 10ns.

The active-high clock gating hold check requires that the gating signal changes only after the falling edge of the clock. Here is the hold path report.

```

Startpoint: UDFF0
  (rising edge-triggered flip-flop clocked by CLKA)
Endpoint: UAND0
  (rising clock gating-check end-point clocked by CLKB)
Path Group: **clock_gating_default**
Path Type: min

```

Point	Incr	Path

clock CLKA (rise edge)	0.00	0.00
clock source latency	0.00	0.00
CLKA (in)	0.00	0.00 r
UDFF0/CK (DF)	0.00	0.00 r
UDFF0/Q (DF)	0.13	0.13 r
UAND0/A1 (AN2)	0.00	0.13 r
data arrival time		0.13
clock CLKB (fall edge)	5.00	5.00
clock source latency	0.00	5.00
CLKB (in)	0.00	5.00 f
UAND0/A2 (AN2)	0.00	5.00 f
clock gating hold time	0.00	5.00
data required time		5.00

data required time		5.00
data arrival time		-0.13

slack (VIOLATED)		-4.87

The hold gating check fails because the gating signal is changing too fast, before the falling edge of *CLKB* at 5ns. If a 5ns delay was added between *UDDF0/Q* and *UAND0/A1* pins, both setup and hold gating checks would pass validating that the gating signal changes only in the specified window.

One can see that the hold time requirement is quite large. This is caused by the fact that the sense of the gating signal and the flip-flops being gated are the same. This can be resolved by using a different type of launch flip-flop, say, a negative edge-triggered flip-flop to generate the gating signal. Such an example is shown next.

In Figure 10-14, the flip-flop *UFF0* is controlled by the negative edge of clock *CLKA*. Safe clock gating implies that the output of flip-flop *UFF0* must change during the inactive part of the gating clock, which is between 5ns and 10ns.

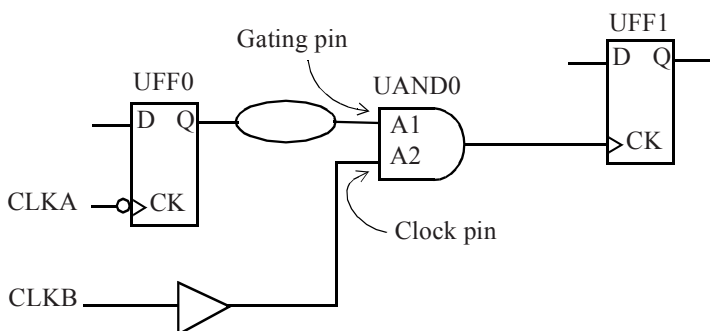


Figure 10-14 *Gating signal clocked on falling edge.*

The signal waveforms corresponding to the schematic in Figure 10-14 are depicted in Figure 10-15. Here is the clock gating setup report.

```
Startpoint: UFF0
(falling edge-triggered flip-flop clocked by CLKA)
Endpoint: UAND0
(rising clock gating-check end-point clocked by CLKB)
```

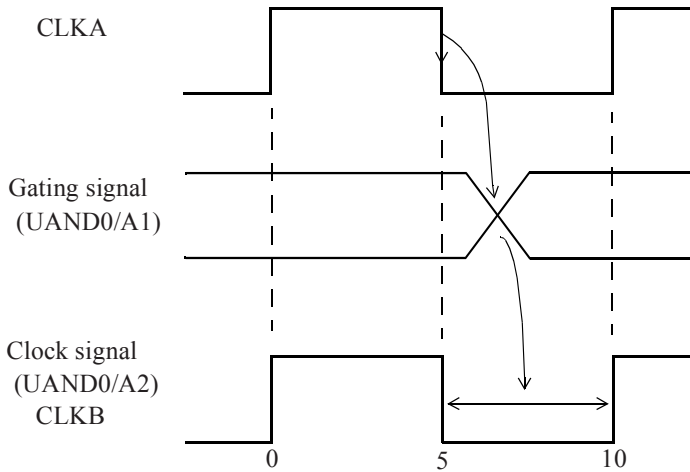


Figure 10-15 *Gating signal generated from negative edge flip-flop meets the gating checks.*

Path Group: ****clock_gating_default****
 Path Type: max

Point	Incr	Path

clock CLKA (fall edge)	5.00	5.00
clock source latency	0.00	5.00
CLKA (in)	0.00	5.00 f
UFF0/CKN (DFN)	0.00	5.00 f
UFF0/Q (DFN)	0.15	5.15 r
UAND0/A1 (AN2)	0.00	5.15 r
data arrival time		5.15
 clock CLKB (rise edge)	 10.00	 10.00
clock source latency	0.00	10.00
CLKB (in)	0.00	10.00 r
UAND0/A2 (AN2)	0.00	10.00 r
clock gating setup time	0.00	10.00
data required time		10.00

data required time		10.00

data arrival time	-5.15

slack (MET)	4.85

Here is the clock gating hold report. Notice that the hold time check is much easier to meet with the new design.

```
Startpoint: UFF0
(falling edge-triggered flip-flop clocked by CLKA)
Endpoint: UAND0
(rising clock gating-check end-point clocked by CLKB)
Path Group: **clock_gating_default**
Path Type: min
```

Point	Incr	Path

clock CLKA (fall edge)	5.00	5.00
clock source latency	0.00	5.00
CLKA (in)	0.00	5.00 f
UFF0/CKN (DFN)	0.00	5.00 f
UFF0/Q (DFN)	0.13	5.13 f
UAND0/A1 (AN2)	0.00	5.13 f
data arrival time		5.13
clock CLKB (fall edge)	5.00	5.00
clock source latency	0.00	5.00
CLKB (in)	0.00	5.00 f
UAND0/A2 (AN2)	0.00	5.00 f
clock gating hold time	0.00	5.00
data required time		5.00

data required time		5.00
data arrival time		-5.13

slack (MET)		0.13

Since the clock edge (negative edge) that launches the gating signal is opposite of the clock being gated (active-high), the setup and hold require-

ments are easy to meet. This is the most common structure used for gated clocks.

Active-Low Clock Gating

Figure 10-16 shows an example of an active-low clock gating check.

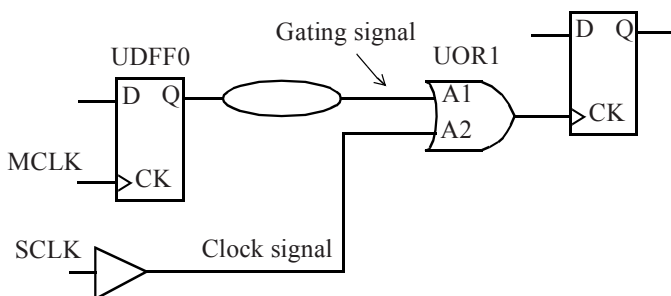


Figure 10-16 *Active-low clock gating check.*

```
create_clock -name MCLK -period 8 -waveform {0 4} \
[get_ports MCLK]
create_clock -name SCLK -period 8 -waveform {0 4} \
[get_ports SCLK]
```

Active-low clock gating check validates that the rising edge of the gating signal arrives at the active portion of the clock (when it is high) for positive edge-triggered logic. As described before, the key is that the gating signal should not cause an active edge for the output gated clock. When the gating signal is high, the clock cannot go through. Thus the gating signal should switch only when the clock is high as illustrated in Figure 10-17.

Here is the active-low clock gating setup timing report. This check ensures that the gating signal arrives before the clock edge becomes inactive, in this case, at 4ns.

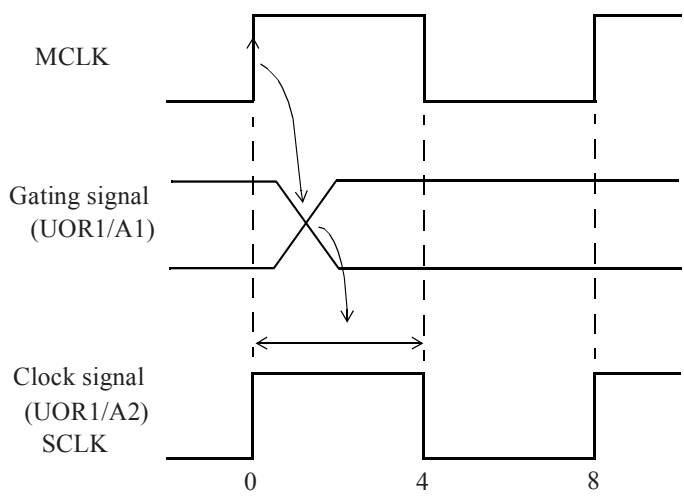


Figure 10-17 *Gating signal changes when clock is high.*

Startpoint: UDF0
(rising edge-triggered flip-flop clocked by MCLK)
Endpoint: UOR1
(**falling clock gating-check** end-point clocked by SCLK)
Path Group: ****clock_gating_default****
Path Type: max

Point	Incr	Path

clock MCLK (rise edge)	0.00	0.00
clock source latency	0.00	0.00
MCLK (in)	0.00	0.00 r
UDF0/CK (DF)	0.00	0.00 r
UDF0/Q (DF)	0.13	0.13 f
UOR1/A1 (OR2)	0.00	0.13 f
data arrival time		0.13
clock SCLK (fall edge)	4.00	4.00
clock source latency	0.00	4.00
SCLK (in)	0.00	4.00 f
UOR1/A2 (OR2)	0.00	4.00 f
clock gating setup time	0.00	4.00

data required time	4.00

data required time	4.00
data arrival time	-0.13

slack (MET)	3.87

Here is the clock gating hold timing report. This check ensures that the gating signal changes only after the rising edge of the clock signal, which in this case is at 0ns.

Startpoint: UDFF0

(rising edge-triggered flip-flop clocked by MCLK)

Endpoint: UOR1

(**falling clock gating-check** end-point clocked by SCLK)

Path Group: ****clock_gating_default****

Path Type: min

Point	Incr	Path
-----		-----
clock MCLK (rise edge)	0.00	0.00
clock source latency	0.00	0.00
MCLK (in)	0.00	0.00 r
UDFF0/CK (DF)	0.00	0.00 r
UDFF0/Q (DF)	0.13	0.13 r
UOR1/A1 (OR2)	0.00	0.13 r
data arrival time		0.13
 clock SCLK (rise edge)	 0.00	 0.00
clock source latency	0.00	0.00
SCLK (in)	0.00	0.00 r
UOR1/A2 (OR2)	0.00	0.00 r
clock gating hold time	0.00	0.00
data required time		0.00
-----		-----
data required time		0.00
data arrival time		-0.13
-----		-----
slack (MET)		0.13

Clock Gating with a Multiplexer

Figure 10-18 shows an example of clock gating using a multiplexer cell. A clock gating check at the multiplexer inputs ensures that the multiplexer select signal arrives at the right time to cleanly switch between *MCLK* and *TCLK*. For this example, we are interested in switching to and from *MCLK* and assume that *TCLK* is low when the select signal switches. This implies that the select signal of the multiplexer should switch only when *MCLK* is low. This is similar to the active-high clock gating check.

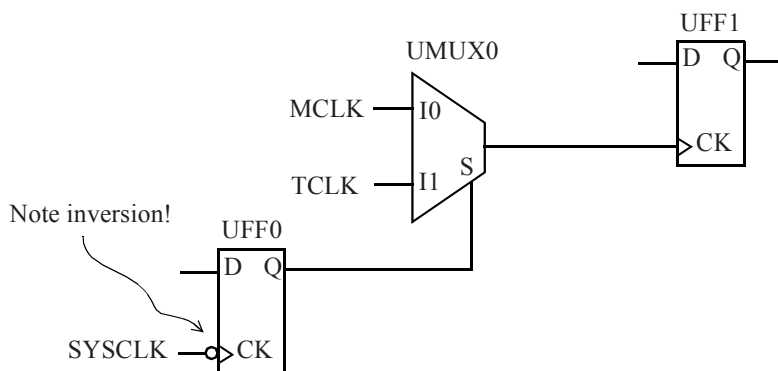


Figure 10-18 *Clock gating using a multiplexer.*

Figure 10-19 shows the timing relationships. The select signal for the multiplexer must arrive at the time *MCLK* is low. Also, assume *TCLK* will be low when select changes.

Since the gating cell is a multiplexer, the clock gating check is not inferred automatically, as evidenced in this message reported during STA.

Warning: No clock-gating check is inferred for clock MCLK at pins UMUX0/S and UMUX0/I0 of cell UMUX0.

Warning: No clock-gating check is inferred for clock TCLK at pins UMUX0/S and UMUX0/I1 of cell UMUX0.

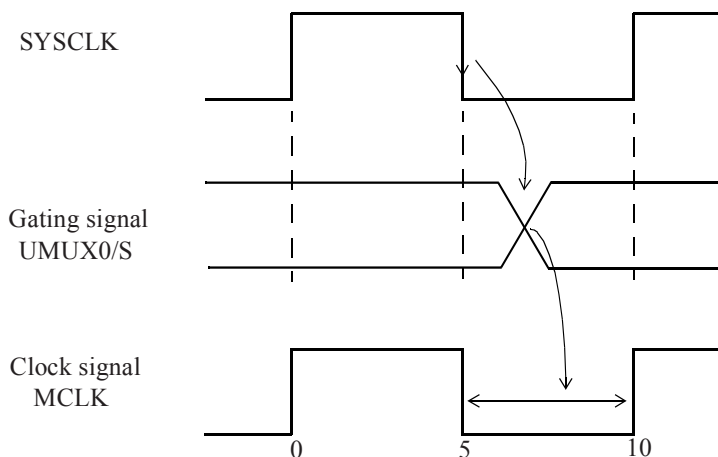


Figure 10-19 *Gating signal arrives when clock is low.*

However a clock gating check can be explicitly forced by providing a `set_clock_gating_check` specification.

```
set_clock_gating_check -high [get_cells UMUX0]
# The -high option indicates an active-high check.
set_disable_clock_gating_check UMUX0/I1
```

The disable check turns off the clock gating check on the specific pin, as we are not concerned with this pin. The clock gating check on the multiplexer has been specified to be an active-high clock gating check. Here is the setup timing path report.

```
Startpoint: UFF0
(falling edge-triggered flip-flop clocked by SYSCLK)
Endpoint: UMUX0
(rising clock gating-check end-point clocked by MCLK)
Path Group: **clock_gating_default**
Path Type: max
```

Point	Incr	Path

clock SYSCLK (fall edge)	5.00	5.00
clock source latency	0.00	5.00
SYSCLK (in)	0.00	5.00 f
UFF0/CKN (DFN)	0.00	5.00 f
UFF0/Q (DFN)	0.15	5.15 r
UMUX0/S (MUX2)	0.00	5.15 r
data arrival time		5.15
clock MCLK (rise edge)	10.00	10.00
clock source latency	0.00	10.00
MCLK (in)	0.00	10.00 r
UMUX0/I0 (MUX2)	0.00	10.00 r
clock gating setup time	0.00	10.00
data required time		10.00

data required time		10.00
data arrival time		-5.15

slack (MET)		4.85

The clock gating hold timing report is next.

Startpoint: UFF0
 (falling edge-triggered flip-flop clocked by SYSCLK)
Endpoint: UMUX0
 (**rising clock gating-check** end-point clocked by MCLK)
Path Group: ****clock_gating_default****
Path Type: min

Point	Incr	Path

clock SYSCLK (fall edge)	5.00	5.00
clock source latency	0.00	5.00
SYSCLK (in)	0.00	5.00 f
UFF0/CKN (DFN)	0.00	5.00 f
UFF0/Q (DFN)	0.13	5.13 f
UMUX0/S (MUX2)	0.00	5.13 f
data arrival time		5.13
clock MCLK (fall edge)	5.00	5.00

clock source latency	0.00	5.00
MCLK (in)	0.00	5.00 f
UMUX0/I0 (MUX2)	0.00	5.00 f
clock gating hold time	0.00	5.00
data required time		5.00

data required time		5.00
data arrival time		-5.13

slack (MET)		0.13

Clock Gating with Clock Inversion

Figure 10-20 shows another clock gating example where the clock to the flip-flop is inverted and the output of the flip-flop is the gating signal. Since the gating cell is an *and* cell, the gating signal must switch only when the clock signal at the *and* cell is low. This defines the setup and hold clock gating checks.

Here is the clock gating setup timing report.

```

Startpoint: UDFFO
(rising edge-triggered flip-flop clocked by MCLK')
Endpoint: UAND0
(rising clock gating-check end-point clocked by MCLK')
Path Group: **clock_gating_default**
Path Type: max

```

Point	Incr	Path

clock MCLK' (rise edge)	5.00	5.00
clock source latency	0.00	5.00
MCLK (in)	0.00	5.00 f
UINV0/ZN (INV)	0.02	5.02 r
UDFF0/CK (DF)	0.00	5.02 r
UDFF0/Q (DF)	0.13	5.15 f
UAND0/A1 (AN2)	0.00	5.15 f
data arrival time		5.15
 clock MCLK' (rise edge)	 15.00	 15.00

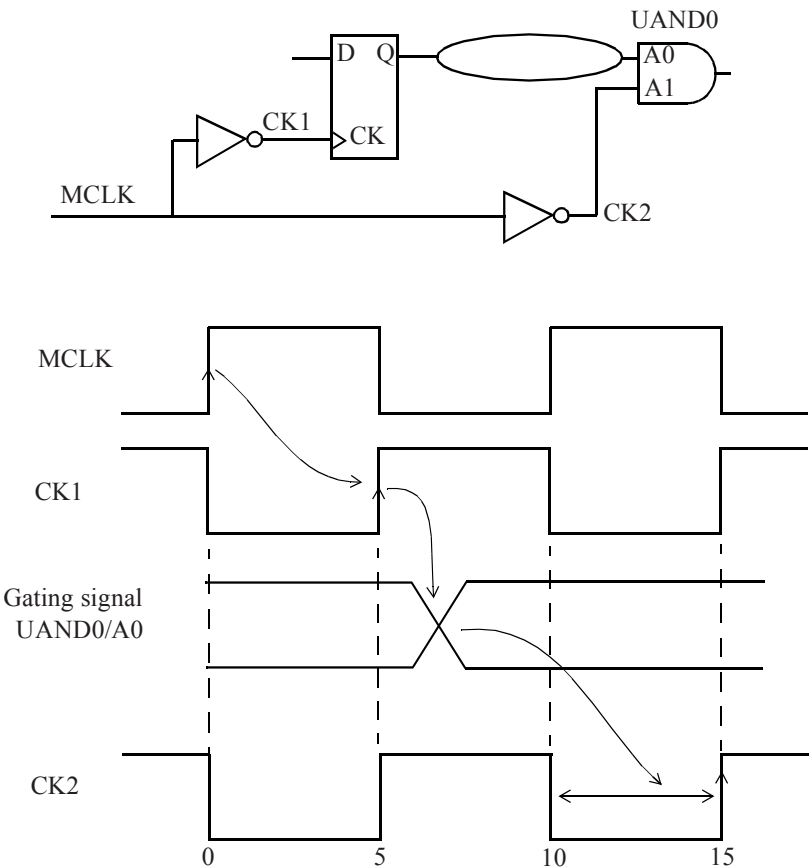


Figure 10-20 *Clock gating example with clock inversion.*

clock source latency	0.00	15.00
MCLK (in)	0.00	15.00 f
UINV1/ZN (INV)	0.02	15.02 r
UAND0/A2 (AN2)	0.00	15.02 r
clock gating setup time	0.00	15.02
data required time		15.02

data required time		15.02

data arrival time	-5.15

slack (MET)	9.87

Notice that the setup check validates if the data changes before the edge on MCLK at time 15ns. Here is the clock gating hold timing report.

```

Startpoint: UDFFO
  (rising edge-triggered flip-flop clocked by MCLK')
Endpoint: UAND0
  (rising clock gating-check end-point clocked by MCLK')
Path Group: **clock_gating_default**
Path Type: min

```

Point	Incr	Path

clock MCLK' (rise edge)	5.00	5.00
clock source latency	0.00	5.00
MCLK (in)	0.00	5.00 f
UINV0/ZN (INV)	0.02	5.02 r
UDFF0/CK (DF)	0.00	5.02 r
UDFF0/Q (DF)	0.13	5.15 r
UAND0/A1 (AN2)	0.00	5.15 r
data arrival time		5.15
 clock MCLK' (fall edge)	 10.00	 10.00
clock source latency	0.00	10.00
MCLK (in)	0.00	10.00 r
UINV1/ZN (INV)	0.01	10.01 f
UAND0/A2 (AN2)	0.00	10.01 f
clock gating hold time	0.00	10.01
data required time		10.01

data required time		10.01
data arrival time		-5.15

slack (VIOLATED)		-4.86

The hold check validates whether the data (gating signal) changes before the falling edge of *MCLK* at time 10ns.

In the event that the gating cell is a complex cell and the setup and hold checks are not obvious, the *set_clock_gating_check* command can be used to specify a setup and hold check on the gating signal that gates a clock signal. The setup check validates that the gating signal is stable before the active edge of the clock signal. A setup failure can cause a glitch to appear at the gating cell output. The hold check validates that the gating signal is stable at the inactive edge of the clock signal. Here are some examples of the *set_clock_gating_check* specification.

```
set_clock_gating_check -setup 2.4 -hold 0.8 \  
  [get_cells U0/UXOR1]  
# Specifies the setup and hold time for the clock  
# gating check at the specified cell.  
set_clock_gating_check -high [get_cells UMUX5]  
# Check is performed on high level of clock. Alternately, the  
-low option can be used for an active-low clock gating check.
```

10.6 Power Management

Managing the power is an important aspect of any design and how it is implemented. During design implementation, a designer typically needs to evaluate different approaches for trade-off between speed, power and area of the design.

As described in Chapter 3, the power dissipated in the logic portion of the design is comprised of leakage power and the active power. In addition, the analog macros and the IO buffers (especially those with active termination) can dissipate power which is not activity dependent and is not leakage. In this section, we focus on the tradeoffs for power dissipated in the logic portion of the design.

In general, there are two considerations for managing the power contributions from the digital logic comprised of standard cells and memory macros:

- *To minimize the total active power of the design.* A designer would ensure that the total power dissipation stays within the available power limit. There may be different limits for different operating modes of the design. In addition, there can also be different limits from different power supplies used in the design.
- *To minimize the power dissipation of the design in standby mode.* This is an important consideration for battery operated devices (for example, cell phone) where the goal is to minimize the power dissipation in standby mode. The power dissipation in standby mode is leakage power plus any power dissipation for the logic that is active in standby mode. As discussed above, there may be other modes such as *sleep mode*, with different constraints on power.

This section describes various approaches for power management. Each of these approaches has its pros and cons which are described herein.

10.6.1 Clock Gating

As described in Chapter 3, clock activity at the flip-flops contributes to a significant component of the total power. A flip-flop dissipates power due to clock toggle even when the flip-flop output does not switch. Consider the example in Figure 10-21(a) where the flip-flop receives new data only when the enable signal *EN* is active otherwise it retains the previous state. During the time *EN* signal is inactive, the clock toggling at the flip-flop do not cause any output change though the clock activity still results in the power dissipated inside the flip-flop. The purpose of clock gating is to minimize this contribution by eliminating the clock activity at the flip-flop during clock cycles when the flip-flop input is not active. The logic restructuring through clock gating introduces gating of the clock at the flip-flop pin. An example of the transformation due to clock gating is illustrated in Figure 10-21.

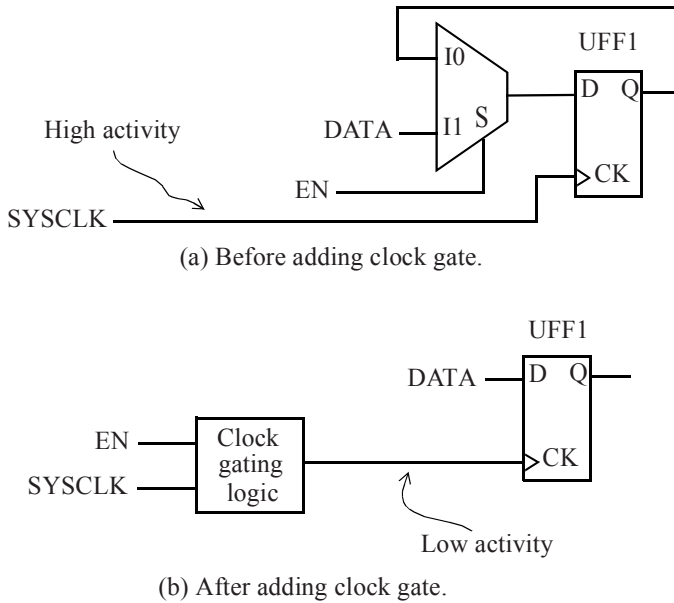


Figure 10-21 Adding clock gate to save flip-flop power.

The clock gating thus ensures that the clock pin of the flip-flop toggles only when new data is available at its data input.

10.6.2 Power Gating

Power gating involves gating off the power supply so that the power to the inactive blocks can be turned off. This procedure is illustrated in Figure 10-22, where a **footer** (or a **header**) MOS device is added in series with the power supply. The control signal *SLEEP* is configured so that the footer (or header) MOS device is *on* during normal operation of the block. Since the power gating MOS device (footer or header) is *on* during normal operation, the block is powered and it operates in normal functional mode. During inactive (or sleep) mode of the block, the gating MOS device (footer or header) is turned off which eliminates any active power dissipation in the logic block. The footer is a large NMOS device between the actual ground and

the ground net of the block which is controlled through power gating. The header is a large PMOS device between the actual power supply and the power supply net of the block which is controlled through power gating. During sleep mode, the only power dissipated in the block is the leakage through the footer (or header) device.

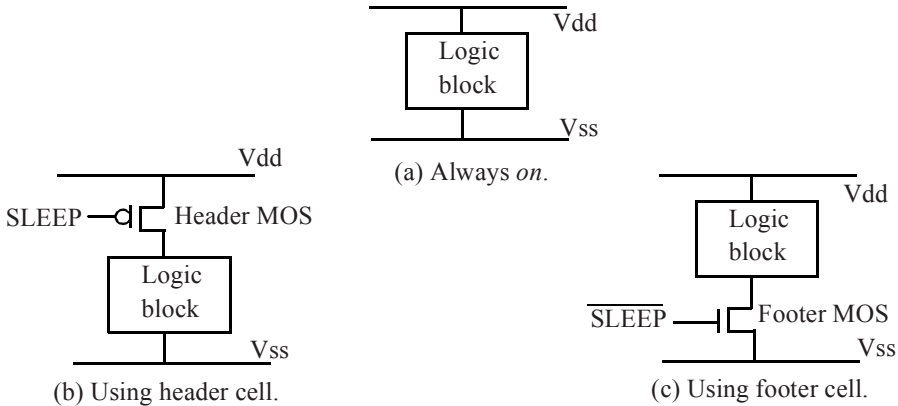


Figure 10-22 Cutting off power to an inactive logic block using a header or a footer device.

The footers or headers are normally implemented using multiple power gating cells which correspond to multiple MOS devices in parallel. The footer and header devices introduce a series *on* resistance to the power supply. If the value of the *on* resistance is not small, the IR drop through the gating MOS device can affect the timing of the cells in the logic block. While the primary criteria regarding the size of the gating devices is to ensure that the *on* resistance value is small, there is a trade-off as the power gating MOS devices determine the leakage in the inactive or sleep mode.

In summary, there should be adequate number of power gating cells in parallel to ensure minimal IR drop from the series *on* resistance in active mode. However, the leakage from the gating cells in the inactive or sleep mode is also a criteria in choosing the number of power gating cells in parallel.

10.6.3 Multi Vt Cells

As described in Chapter 3 (Section 3.8), the multi Vt cells are used to trade-off speed with leakage. The high Vt cells have less leakage, though these are slower than the standard Vt cells which are faster but have higher leakage. Similarly, the low Vt cells are faster than standard Vt cells but the leakage is also correspondingly higher.

In most designs, the goal is to minimize the total power while achieving the desired operational speed. Even though leakage can be a significant component of the total power, implementing a design with only high Vt cells to reduce leakage can increase the total power even though the leakage contribution may be reduced. This is because the resulting design implementation may require many more (or higher strength) high Vt cells to achieve the required performance. The increase in equivalent gate count can increase the active power much more than the reduction in leakage power due to use of high Vt cells. However, there are scenarios where the leakage is a dominant component of the total power; in such cases, a design with high Vt cells can result in reduction of the total power. The above trade-off between cells with different Vt in terms of their speed and leakage needs to be utilized suitably since it is dependent on the design and its switching activity profile. Two scenarios of a high performance block are illustrated below where the implementation approach can be different depending on whether the block is very active or has low switching activity.

High Performance Block with High Activity

This scenario is of a high performance block with high switching activity and the power is dominated by the active power. For such blocks, focusing only on reducing leakage power can cause the total power to increase even though the leakage contribution may be minimized. In such cases, the initial design implementation should use standard Vt (or low Vt) cells to meet the desired performance. After the required timing is achieved, the cells along the paths which have positive timing slack can be changed into high Vt cells so that the leakage contribution is reduced while still meeting the timing requirement. Thus, in the final implementation, the standard Vt (or low Vt) cells are used only along the critical or hard to achieve timing

paths, whereas cells along the non-critical timing paths can be high V_t cells.

High Performance Block with Low Activity

This scenario is of a high performance block with very low switching activity so that the leakage power is a significant component of the total power. Since the block has low activity, the active power is not a major component for the total power of the design. For such blocks, the initial implementation attempts to use only high V_t cells in the combinational logic and flip-flops. An exception is the clock tree which is always active and therefore is built with standard V_t (or low V_t) cells. After the initial implementation with only high V_t cells, there may be some timing paths where the required timing cannot be achieved. The cells along such paths are then replaced with standard V_t (or low V_t) cells to achieve the required timing performance.

10.6.4 Well Bias

The **well bias** refers to adding a small voltage bias to the P-well or N-well used for the NMOS and PMOS devices respectively. The bulk (or P-well) connection for the NMOS device shown in Figure 2-1 is normally connected to the ground. Similarly, the bulk (or N-well) connection for the PMOS device shown in Figure 2-1 is normally connected to the power (V_{dd}) rail.

The leakage power can be reduced significantly if the well connections have a slight negative bias. This means that the P-well for the NMOS devices is connected to a small negative voltage (such as $-0.5V$). Similarly, the N-well connection for the PMOS devices is connected to a voltage above the power rail (such as $V_{dd} + 0.5V$). By adding a well bias, the speed of the cell is impacted; however the leakage is reduced substantially. The timing in the cell libraries are generated by taking the well bias into account.

The drawback of using well bias is that it requires additional supply levels (such as $-0.5V$ and $V_{dd}+0.5V$) for the P-well and N-well connections.

10.7 Backannotation

10.7.1 SPEF

How does STA know what the parasitics of the design are? Quite often, this information is extracted by using a parasitic extraction tool and this data is read in the form of SPEF by the STA tool. Detailed information and the format of the SPEF are described in Appendix C.

An STA engine inside a physical design layout tool also behaves similarly, except that the extraction information is written to an internal database.

10.7.2 SDF

In some cases, the delays of the cells and interconnect are computed by another tool and these are read in for STA via SDF. The advantage of using SDF is that the cell delays and interconnect delays no longer need to be computed - as these come from the SDF directly, and consequently STA can focus on the timing checks. However, the disadvantage of this delay annotation is that STA cannot perform crosstalk computation as the parasitic information is missing. SDF is the mechanism normally used to pass delay information to simulators.

Detailed information and the format of SDF are described in Appendix B.

10.8 Sign-off Methodology

STA can be run for many different scenarios. The three main variables that determine a scenario are:

- Parasitics corners (RC interconnect corners and operating conditions used for parasitic extraction)
- Operating mode
- PVT corner

Parasitic Interconnect Corners

Parasitics can be extracted at many corners. These are mostly governed by the variations in the metal width and metal etch in the manufacturing process. Some of these are:

- *Typical*: This refers to the nominal values for interconnect resistance and capacitance.
- *Max C*: This refers to the interconnect corner which results in maximum capacitance. The interconnect resistance is smaller than at *typical* corner. This corner results in largest delay for paths with short nets and can be used for max path analysis.
- *Min C*: This refers to the interconnect corner which results in minimum capacitance. The interconnect resistance is larger than at *typical* corner. This corner results in smallest delay for paths with short nets and can be used for min path analysis.
- *Max RC*: This refers to the interconnect corner which maximizes the interconnect RC product. This typically corresponds to larger etch which reduces the trace width. This results in largest resistance but corresponds to smaller than typical capacitance. Overall, this corner has the largest delay for paths with long interconnects and can be used for max path analysis.
- *Min RC*: This refers to the interconnect corner which minimizes the interconnect RC product. This typically corresponds to smaller etch which increases the trace width. This results in smallest resistance but corresponds to larger than typical capacitance. Overall, this corner has the smallest path delay for paths with long interconnects and can be used for min path analysis.

Based upon the interconnect R and C for various corners described above, an interconnect corner with larger C results in smaller R and a corner with smaller C results in larger R. Thus, the R partially compensates for the C across various interconnect corners. This implies that no single corner maps to an extreme value (worst-case or best-case) for path delay for all types of nets. The path delay using C_{worst} / C_{best} corners is extreme only for short nets while RC_{worst} / RC_{best} corners is extreme only for long nets. The *typical* interconnect corner is often the extreme in terms of path delay

for nets with average length. Thus, designers often choose to verify the timing at various interconnect corners described above. However, even the verification at each corner does not cover all possible scenarios since different metal layers can actually be at different interconnect corners independently - for example, *Max C* corner for *METAL2*, *Max RC* corner for *METAL1*, and so on. Statistical timing analysis described in Section 10.9 offers a mechanism for static timing analysis where different metal layers can be at different interconnect corners.

Operating Modes

The operating mode dictates the operation of the design. Various operating modes for a design can be:

- Functional mode 1 (for e.g. high-speed clocks)
- Functional mode 2 (for e.g. slow clocks)
- Functional mode 3 (for e.g. sleep mode)
- Functional mode 4 (for e.g. debug mode)
- Test mode 1 (for e.g. scan capture mode)
- Test mode 2 (for e.g. scan shift mode)
- Test mode 3 (for e.g. bist mode)
- Test mode 4 (for e.g. jtag mode).

PVT Corners

The PVT corners dictate at what conditions the STA analysis takes place. The most common PVT corners are:

- WCS (slow process, low power supply, high temperature)
- BCF (fast process, high power supply, low temperature)
- Typical (typical process, nominal power supply, nominal temperature)
- WCL (worst-case slow at cold - slow process, low power supply, low temperature)
- Or any other point in the PVT domain.

STA analysis can be performed for any scenario. A scenario here refers to a combination of the interconnect corner, operating mode, and PVT corner described above.

Multi-Mode Multi-Corner Analysis

Multi-mode multi-corner (MMMC) analysis refers to performing STA across multiple operating modes, PVT corners and parasitic interconnect corners at the same time. For example, consider a DUA that has four operating modes (*Normal*, *Sleep*, *Scan shift*, *Jtag*), and is being analyzed at three PVT corners (*WCS*, *BCF*, *WCL*) and three parasitic interconnect corners (*Typical*, *Min C*, *Min RC*) as shown in Table 10-23._

<i>PVT corner/ Parasitic corner</i>	<i>WCS</i>	<i>BCF</i>	<i>WCL</i>
Typical	1: Normal/Sleep/ Scan shift/ Jtag	2: Normal/ Sleep/ Scan shift	3: Normal/ Sleep
Min C	4: Not required	5: Normal/ Sleep	6: Not required
Min RC	7: Not required	8: Normal/ Sleep	9: Not required

Table 10-23 *Multiple modes and corners used during timing sign-off.*

There are a total of thirty six possible scenarios at which all timing checks, such as setup, hold, slew, and clock gating checks can be performed. Running STA for all thirty six scenarios at the same time can be prohibitive in terms of runtime depending upon the size of the design. It is possible that a scenario may not be necessary as it may be included within another scenario, or a scenario may not be required. For example, the designer may determine that scenarios 4, 6, 7 and 9 are not relevant and thus are not required. Also, it may not be necessary to run all modes in one corner, such as *Scan shift* or *Jtag* modes may not be needed in scenario 5. STA could be run on a single scenario or on multiple scenarios concurrently if multi-mode multi-corner capability is available.

The advantage of running multi-mode multi-corner STA is of savings in runtime and complexity in setting up the analysis scripts. Additional savings in an MMMC scenario is that the design and parasitics need to be loaded only once or twice as opposed to loading these individually multiple times for each mode or corner. Such a job is also more amenable to running them on an LSF farm. Multi-mode multi-corner has a bigger advantage in an optimization flow where the optimization is done across all scenarios such that fixing timing violations in one scenario does not introduce timing violations in another scenario.

For IO constraints, the *-add_delay* option can be used with multiple clock sources to analyze different modes in one run, such as scan or bist modes, or different operating modes in a PHY¹ corresponding to different speeds. Often each mode is analyzed in a separate run, but not always.

It is not unusual to find a design with a large number of clocks that requires tens of independent runs to cover every mode in max and min corners, and including the effect of crosstalk and noise.

10.9 Statistical Static Timing Analysis

The static timing analysis techniques described thus far are deterministic since the analysis is based upon fixed delays for all timing arcs in the design. The delay of each arc is computed based upon the operating conditions along with the process and interconnect models. While there may be multiple modes and multiple corners, the timing path delays for a given scenario are obtained deterministically.

In practice, the WCS or BCF for the process and operating corner conditions typically used during the STA correspond to the extreme 3σ corners². The timing libraries are based upon the process corner models provided by the foundry and characterized with the operating conditions which result

1. Physical layer interface IP block such as a 10G PHY.

2. The σ here refers to standard deviation of an independent variable modeled statistically.

in the corresponding corner for the timing values of the cells. For example, the best-case fast library is characterized using fast process models, highest power supply and lowest temperature.

10.9.1 Process and Interconnect Variations

Global Process Variations

The global process variations, which are also called **inter-die device variations**, refer to the variations in the process parameters which impact all devices on a die (or wafer). See Figure 10-24. This depicts that all devices on a die are impacted similarly by these process variations - every device on a die will be *slow* or *fast* or anywhere in between. Thus, the variations modeled by the global process parameters are intended to capture the variations from die to die.

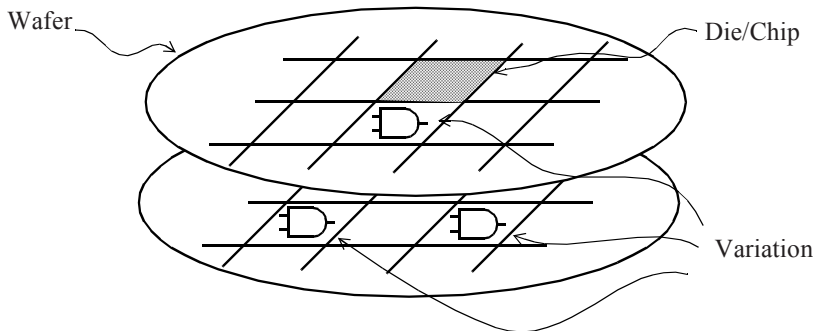


Figure 10-24 *Inter-die process variations.*

An illustration of the variations of a global parameter value (say g_par1) is shown in Figure 10-25. For example, the parameter g_par1 may correspond to IDS_{sat} (device saturation current) for a standard¹ NMOS device. Since this is a global parameter, all NMOS devices in all cell instances of a die will correspond to the same value of g_par1 . This can alternately be stated

1. The standard device here means a device with fixed length and width.

as follows. The variations in g_par1 for all cell instances are fully correlated or the variations in g_par1 on a die track each other. Note that there would be other global parameters ($g_par2, \dots$) which may, for example, model the PMOS device saturation current and other relevant variables.

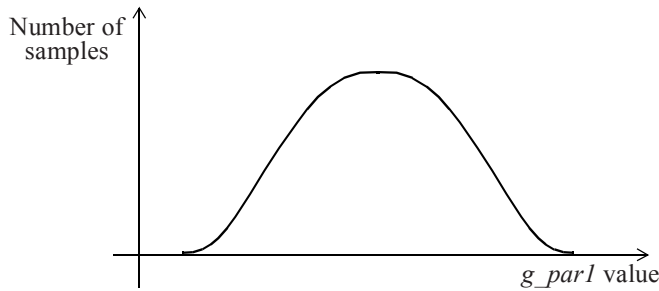


Figure 10-25 *Variations in a global parameter.*

Different global parameters ($g_par1, g_par2, \dots$) are uncorrelated. The variations in different global parameters do not track each other which means that the g_par1 and g_par2 parameters vary independently of each other; in a die g_par1 may be at its maximum while g_par2 may be at its minimum.

In the deterministic (that is, non-statistical) analysis, the slow process models may correspond to the $+3\sigma$ corner condition for the inter-die variations. Similarly, the fast process models may correspond to the -3σ corner condition for the inter-die variations.

Local Process Variations

The local process variations, which are also called **intra-die device variations**, refer to the variations in the process parameters which can affect the devices differently on a given die. See Figure 10-26. This implies that identical devices on a die placed side by side may have different behavior on the same die. The variations modeled by the local process variations are intended to capture the random process variations within the die.

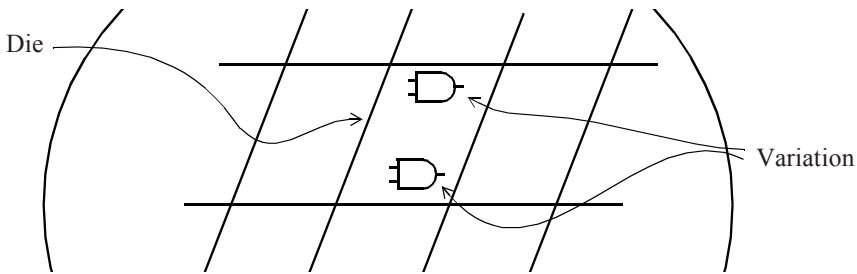


Figure 10-26 *Intra-die device variation.*

An illustration of the variations in a local process parameter is depicted in Figure 10-27. The local parameter variations on a die do not track each other and their variations from one cell instance to another cell instance are uncorrelated. This means that a local parameter may have different values for different devices on the same die. For example, different NAND2 cell instances on a die may see different local process parameter values. This can cause different instances of the same NAND2 cell to have different delay values even if other parameters such as input slew and output loading are identical.

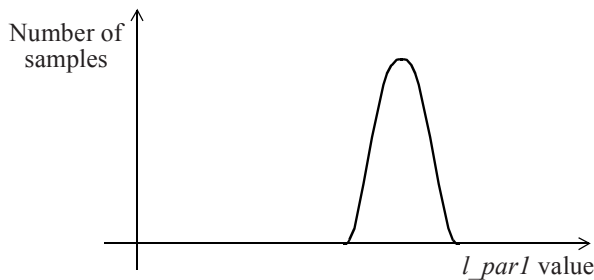


Figure 10-27 *Variations in local process parameter.*

An illustration of the variations in the NAND2 cell delay caused by global and local variations is depicted in Figure 10-28. The figure illustrates that the global parameter variations cause larger delay variation than the local parameter variations.

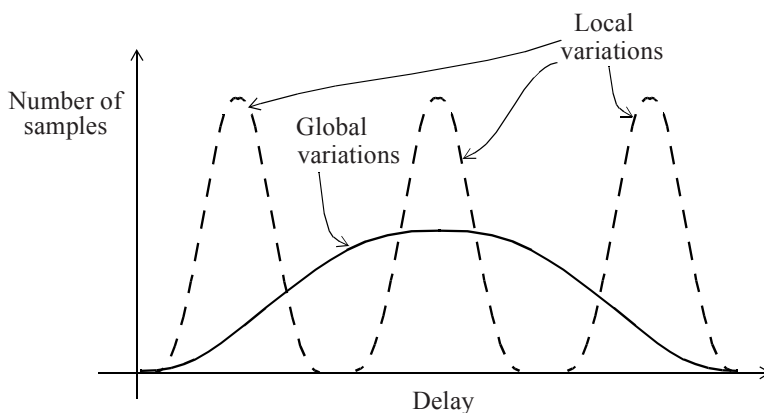


Figure 10-28 Variation in cell delay due to global and local process variations.

The local process variations are one of the variations intended to be captured in the analysis using OCV modeling, described in Section 10.1. Since statistical timing models normally include the local process variations, the OCV analysis using statistical timing models should not include the local process variation in the OCV setting.

Interconnect Variations

As described in Section 10.8, there are various interconnect corners which represent the parameter variations of each metal layer affecting the interconnect resistance and capacitance values. These parameter variations are generally the thicknesses of the metal and the dielectric, and the metal etch which affects the width and spacing of the metal traces in various metal layers. In general, the parameters affecting a metal impact the parasitics of all traces in that metal layer but have minimal or no effect on the parasitics of the traces in other metal layers.

The interconnect corners described in Section 10.8 model the interconnect variations so that all the metal layers map to the same interconnect corner. The interconnect variations when modeled statistically allow each metal

layer to vary independently. The statistical approach models all possible combinations of variations in the interconnect space and thus models variations which may not be captured by analyzing only at the specified interconnect corners. For example, it is possible that the launch path of a clock tree is in *METAL2*, whereas the capture path of the clock tree is in *METAL3*. Timing analysis at the traditional interconnect corners considers various corners which vary all metals together and thus cannot model the scenario where the *METAL2* is at a corner which results in max delay, and the *METAL3* is at a corner which results in min delay. Such a combination corresponds to the worst-case scenario for the setup paths and can only be captured by modeling the interconnect variations statistically.

10.9.2 Statistical Analysis

What is SSTA?

The modeling of variations described above is feasible if the cell timing models and the interconnect parasitics are modeled statistically. Apart from delay, the pin capacitance values at the inputs of the cells are also modeled statistically. This implies that the timing models are described in terms of mean and standard deviations with respect to process parameters (global and local). The interconnect resistances and capacitances are described in terms of mean and standard deviations with respect to interconnect parameters. The delay calculation procedures (described in Chapter 5) obtain the delays of each timing arc (cell as well as interconnect) which are then represented by mean and standard deviations with respect to various parameters. Thus, every delay is represented by a mean and N standard deviations (where N is the number of independent process and interconnect parameters modeled statistically).

Since the delays through individual timing arcs are expressed statistically, the statistical static timing analysis (SSTA) procedure combines the delays of the timing arcs to obtain the path delay which is also expressed statistically (with mean and standard deviations). The SSTA maps the standard deviations with respect to the independent process and interconnect parameters to obtain the overall standard deviation of the path delay. For example, consider the path delay comprised from two timing arcs as shown

in Figure 10-29. Since each delay component has its variations, the variations are combined differently depending upon whether these are correlated or uncorrelated. If the variations are from the same source (such as caused by *g_par1* which track each other), the σ of the path delay is simply equal to $(\sigma_1 + \sigma_2)$. However, if the variations are uncorrelated (such as due to *l_par1*), the σ of the path delay is equal to $\sqrt{\sigma_1^2 + \sigma_2^2}$, which is smaller than $(\sigma_1 + \sigma_2)$. The phenomenon of smaller σ for the path delay when modeling local (uncorrelated) process variations is also referred to as statistical cancellation of the individual delay variations.

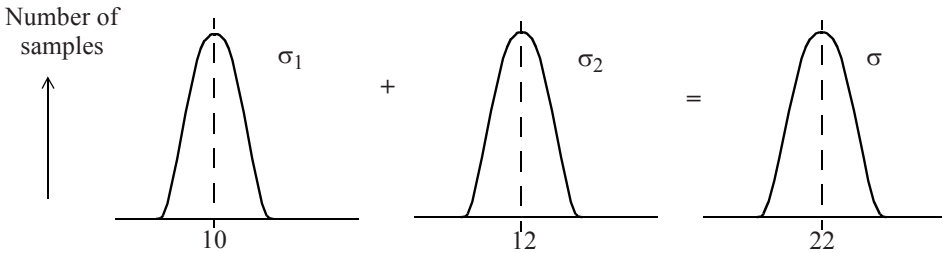


Figure 10-29 Path delay comprised of variations in components.

For a real design, both correlated as well as uncorrelated variations are modeled, and thus the contributions from both of these types of variations need to be combined appropriately.

The clock path delays for launch and capture clock are also expressed statistically in the same manner. Based upon the data and clock path delays, the slack is obtained as a statistical variable with its nominal value as well as standard deviation.

Assuming normal distribution, effective minimum and maximum values corresponding to $(mean \pm 3\sigma)$ can be obtained. The $(mean - 3\sigma)$ corresponds to 0.135% and 99.865% quantile values of the normal distribution shown in Figure 10-30. The 0.135% quantile means that only 0.135% of the resulting distribution is smaller than this value ($mean - 3\sigma$); similarly 99.865% quantile means that 99.865% of the distribution is smaller than this value or only 0.135% ($100\% - 99.865\%$) of the distribution is larger than this

value ($mean + 3\sigma$). The effective lower and upper bounds are referred to as the quantiles in an SSTA report and the designer can select the quantile value used in the analysis, such as 0.5% or 99.5% which corresponds to ($mean -/+ 2.576 \sigma$).

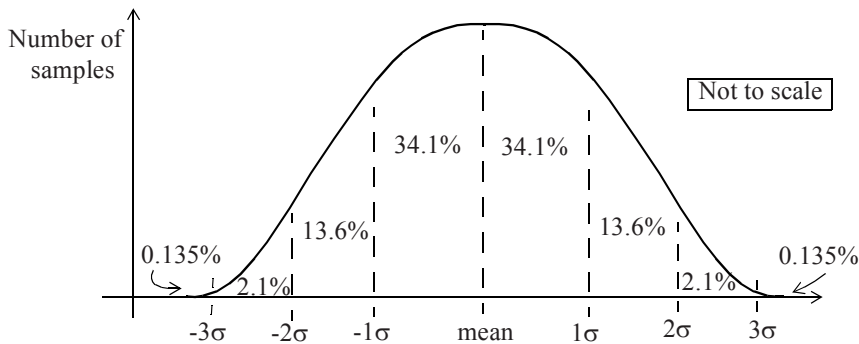


Figure 10-30 Normal distribution.

For noise and crosstalk analysis (Chapter 6), the path delays as well as timing windows used are modeled statistically with mean and standard deviations with respect to various parameters.

Based upon the path slack distribution, the SSTA reports the mean, standard deviation and the quantile values of slack for each path whereby the passing or failing can be determined based upon the required statistical confidence.

Statistical Timing Libraries

In the SSTA approach, the standard cell libraries (and libraries for other macros used in a design) provide timing models at various environmental conditions. For example, the analysis at min V_{dd} and high temperature corner utilizes libraries which are characterized at this condition but the process parameters are modeled statistically. The library includes timing

models for the nominal parameter values as well as with parameter variations. For N process parameters, a statistical timing library characterized at a power supply of 0.9V and 125C may include the following:

- Timing models with nominal process parameters, *plus the following with respect to each of the process parameters.*
- Timing models with respect to a parameter i at $(\text{nominal} + 1\sigma)$, the other parameters being held at nominal value.
- Timing models with respect to a parameter i at $(\text{nominal} - 1\sigma)$, the other parameters being held at nominal value.

For a simplified example scenario with two independent process parameters, the timing models are characterized with the nominal parameter values and also with the variations in parameter values as illustrated in Figure 10-31.

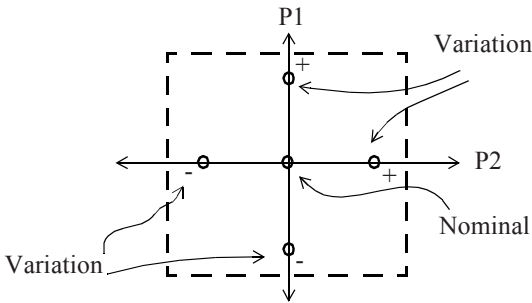


Figure 10-31 *N-dimensional process space.*

Statistical Interconnect Variations

There are three independent parameters for each metal layer:

- *Metal etch.* This controls the metal width as well as spacing to the neighboring conductor. A large etch in a metal layer reduces the width (which increases resistance) and increases the spacing to

the neighboring traces (which reduces the coupling capacitance to the neighboring traces). This parameter is expressed as a variation in the width of the conductor.

- *Metal thickness.* Thicker metal implies larger capacitance to the layers below. This is expressed as a variation in the thickness of the conductor.
- *IMD (Inter Metal Dielectric) thickness.* Larger IMD thickness reduces the coupling to the layers below. This parameter is expressed as a variation in the IMD thickness.

SSTA Results

The output results in the statistical analysis provide path slack in terms of its mean and effective corner values. An example of the SSTA report for a setup check (max path analysis) is shown below.

Path	startpoint	endpoint	quantile	sensitiv	mean	stddev
0	DBUS[7]	PDAT[5]	-0.43	50.00	0.86	0.43

Path attribute	quantile	sensitiv	mean	stddev
arrival	6.74	7.88	5.45	0.43
slack	-0.43	50.00	0.86	0.43
required	6.31	0.00	6.31	0.00
startpoint_clock_latency	0.25	0.00	0.25	0.00
endpoint_clock_latency	0.33	0.00	0.33	0.00

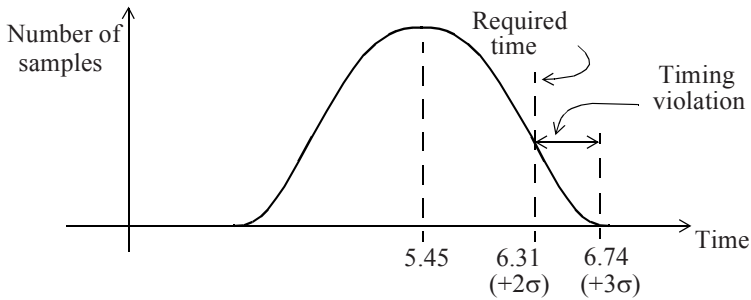
Point arrival	quantile	sensitiv	mean	stddev	incr
DBUS[7]/CP (SDFQD2)	0.25	0.00	0.25	0.00	
DBUS[7]/Q (SDFQD2)	0.66	4.04	0.61	0.02	0.02
U1/ZN (INVD2)	0.80	4.09	0.72	0.03	0.01
...					
U22/ZN (NR3D4)	2.20	5.82	1.89	0.11	0.03
U23/Z (AN2D3)	2.41	6.40	2.03	0.13	0.02
U24/ZN (CKBD3)	2.53	7.10	2.10	0.15	0.02

U25/Z (AO23D2)	2.89	8.65	2.31	0.20	0.05
U26/ZN (IND2D4)	2.98	8.84	2.36	0.21	0.01
U27/Z (MUX3D4)	3.26	8.89	2.58	0.23	0.02
. . .					
U51/ZN (ND2D4)	6.74	7.88	5.45	0.43	0.02
PDAT[5]/D (SDFQD1)	6.74	7.88	5.45	0.43	0.00

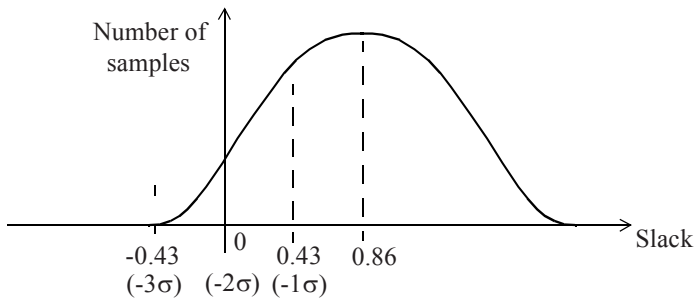
The above report shows that while the mean of the timing path meets the requirement, the 0.135% quantile value has a violation by 0.43ns - path slack quantile is -0.43ns. The path slack has a mean value of +0.86ns with 0.43ns standard deviation. This implies that $\pm 2\sigma$ of the distribution meets the requirement. Since 95.5% of the distribution falls within 2σ variation, this implies that only 2.275% of the manufactured parts will have a timing violation (the remaining 2.275% of the distribution has large positive path slack). A 2.275% quantile setting will thus show a slack of 0 or no timing violation. The arrival time and the path slack distribution is depicted in Figure 10-32.

Note that the above report is for the setup path and thus the quantile column provides the upper bound quantile (for example $+3\sigma$ value for path delay) - the hold path would specify the equivalent lower bound quantile (for example -3σ value). The *sensitiv* column in the report refers to the sensitivity which is the ratio of the standard deviation to the mean (expressed as a percentage). In terms of slack, smaller sensitivity is desired which means that a path passing at mean value continues to pass even with variations. The *incr* column specifies the incremental standard deviation for that line in the report.

With the statistical models for the cells and interconnect, the statistical timing approach analyzes the design at corner environment conditions and explores the space due to *process* and interconnect parameter variations. For example, a statistical analysis at worst-case VT (Voltage and Temperature) would explore the entire global *process* and interconnect space. Another statistical analysis at the best-case VT (Voltage and Temperature) would also explore the entire *process* and interconnect space. These analyses can be contrasted with the traditional corner analysis at the worst-case or the best-case PVT, each of which explores only a single point of process and interconnect space.



(a) Arrival time distribution.



(b) Slack distribution.

Figure 10-32 *Path delay and slack distribution.*

10.10 Paths Failing Timing?

In this section, we provide examples that highlight the critical aspects that a designer needs to focus on during debugging of STA results. Several of these examples contain only the relevant excerpts from the STA reports.

No Path Found

What if one is trying to obtain a path report and the STA reports that no path is found, or it provides a path report but the slack is infinite? In both of these cases, the situation likely occurs because:

- i.* the timing path is broken, or
- ii.* the path does not exist, or
- iii.* there is a false path.

In each of these cases, careful debugging of the constraints is required to identify what constraint causes the path to be blocked. One brute force option is to remove all false path settings and timing breaks and then see if the path can be timed. (A timing break is the removal of a timing arc from STA and is achieved by using the **set_disable_timing** specification as described in Section 7.10.)

Clock Crossing Domain

Here is a header of a path report.

```
Startpoint: IP_IO_RSTHN[0] (input port clocked by SYS_IN_CLK)
Endpoint: X_WR_PTR_GEN/Q_REG
          (recovery check against rising-edge clock PX9_CLK)
Path Group: **async_default**
Path Type: max
```

Point	Incr	Path

clock SYS_IN_CLK (rise edge)	6.00	6.00
. . .		
IO_IO_RSTHN[0] (in)	0.00	6.00 r
. . .		
X_WR_PTR_GEN/Q_REG/CDN (DFCN)	0.00	6.31 r
. . .		
clock PX9_CLK (rise edge)	12.00	12.00
. . .		

<code>library recovery time</code>	<code>0.122</code>	<code>15.98</code>
<code>. . .</code>		

The first thing to notice is that this path starts from an input port and ends at the clear pin of a flip-flop, and a recovery check (see *library recovery time*) on the clear pin is being validated. The next thing to notice is that the path goes across two different clock domains, *SYS_IN_CLK*, the clock that launches the input, and *PX9_CLK*, the clock at the flip-flop whose recovery timing is being checked. Even though it is not apparent from the timing report, but from the design knowledge, one can examine if the two clocks are fully asynchronous and whether any paths between these two clock domains should be treated as false.

Lesson: Verify if the launch clock and capture clock and the paths between the two are valid.

Inverted Generated Clocks

When creating generated clocks, the *-invert* option needs to be used carefully. If a generated clock is specified using the *-invert* option, STA assumes that the generated clock at the specified point is of the type specified. However based upon the logic, it is possible that such a waveform cannot occur in the design. STA would normally provide an error or a warning message indicating that the generated clock is not realizable, however it will continue with the analysis and report the timing paths.

Consider Figure 10-33. Let us define a generated clock with *-invert* on the output of the cell *UCKBUF0*.

```
create_clock -name CLKM -period 10 -waveform {0 5} \
  [get_ports CLKM]
create_generated_clock -name CLKGEN -divide_by 1 -invert \
  -source [get_ports CLKM] [get_pins UCKBUF0/C]
```

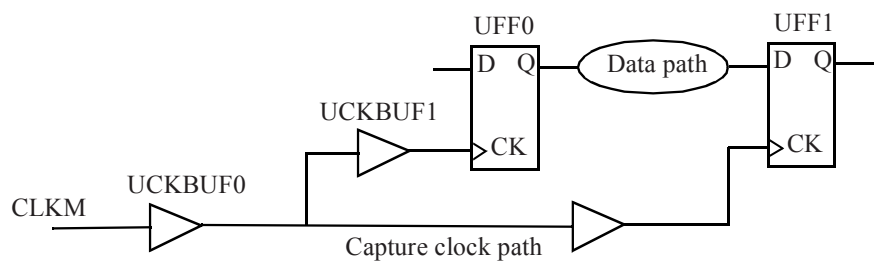


Figure 10-33 *Example of a generated clock.*

Here is the setup timing report based upon the above specifications.

```
Startpoint: UFF0
(rising edge-triggered flip-flop clocked by CLKGEN)
Endpoint: UFF1
(rising edge-triggered flip-flop clocked by CLKGEN)
Path Group: CLKGEN
Path Type: max
```

Point	Incr	Path

clock CLKGEN (rise edge)	5.00	5.00
clock network delay (ideal)	0.00	5.00
UFF0/CK (DF)	0.00	5.00 r
UFF0/Q (DF) <=	0.14	5.14 f
UNOR0/ZN (NR2)	0.04	5.18 r
UBUF4/Z (BUFF)	0.05	5.23 r
UFF1/D (DF)	0.00	5.23 r
data arrival time		5.23
clock CLKGEN (rise edge)	15.00	15.00
clock network delay (ideal)	0.00	15.00
UFF1/CK (DF)		15.00 r
library setup time	-0.05	14.95
data required time		14.95

data required time		14.95
data arrival time		-5.23

slack (MET)

9.72

Notice that the STA faithfully assumes that the waveform at the output of cell *UCKBUF0* is the inverted clock of clock *CLKM*. Thus, the rise edge is at 5ns and the capture setup clock edge is at 15ns. Other than the fact that the rising edge of the clock is at 5ns instead of 0ns, it is not apparent from the timing report that something is wrong. It should be noted that since the error is on the common portion of both the launch and the capture clock paths, the setup and hold timing checks are indeed performed correctly. The warnings and the errors produced by STA need to be carefully analyzed and understood.

The important point to note is that STA will create the generated clock as specified whether it is realizable or not.

Now let us try to move the generated clock with the *-invert* option to the output of the cell *UCKBUF1* and see what happens.

```
create_clock -name CLKM -period 10 -waveform {0 5} \
  [get_ports CLKM]
create_generated_clock -name CLKGEN -divide_by 1 -invert \
  -source [get_ports CLKM] [get_pins UCKBUF1/C]
```

Here is the setup report.

```
Startpoint: UFF0
(rising edge-triggered flip-flop clocked by CLKGEN)
Endpoint: UFF1 (rising edge-triggered flip-flop clocked by CLKM)
Path Group: CLKM
Path Type: max
```

Point	Incr	Path

clock CLKGEN (rise edge)	5.00	5.00
clock network delay (ideal)	0.00	5.00
UFF0/CK (DF)	0.00	5.00 r

UFF0/Q (DF) <-	0.14	5.14 f
UNOR0/ZN (NR2)	0.04	5.18 r
UBUF4/Z (BUFF)	0.05	5.23 r
UFF1/D (DF)	0.00	5.23 r
data arrival time		5.23
clock CLKM (rise edge)	10.00	10.00
clock source latency	0.00	10.00
CLKM (in)	0.00	10.00 r
UCKBUF0/C (CKB)	0.06	10.06 r
UCKBUF2/C (CKB)	0.07	10.12 r
UFF1/CK (DF)	0.00	10.12 r
clock uncertainty	-0.30	9.82
library setup time	-0.04	9.78
data required time		9.78

data required time		9.78
data arrival time		-5.23

slack (MET)		4.55

The path looks like a half-cycle path, but this is incorrect since there is no inversion on the clock path in the actual logic. Once again, STA assumes that the clock at the *UCKBUF1/C* pin is the one as specified in the *create_generated_clock* command. Hence the rising edge occurs at 5ns. The capture clock edge is running off clock *CLKM*, whose next rising edge occurs at 10ns. The hold path report below also contains a similar discrepancy as the setup path.

Startpoint: UFF0		
(rising edge-triggered flip-flop clocked by CLKGEN)		
Endpoint: UFF1 (rising edge-triggered flip-flop clocked by CLKM)		
Path Group: CLKM		
Path Type: min		
Point	Incr	Path

clock CLKGEN (rise edge)	5.00	5.00
clock network delay (ideal)	0.00	5.00
UFF0/CK (DF)	0.00	5.00 r

UFF0/Q (DF) <-	0.14	5.14 r
UNOR0/ZN (NR2)	0.02	5.16 f
UBUF4/Z (BUFF)	0.06	5.21 f
UFF1/D (DF)	0.00	5.21 f
data arrival time		5.21
clock CLKM (rise edge)	0.00	0.00
clock source latency	0.00	0.00
CLKM (in)	0.00	0.00 r
UCKBUF0/C (CKB)	0.06	0.06 r
UCKBUF2/C (CKB)	0.07	0.12 r
UFF1/CK (DF)	0.00	0.12 r
clock uncertainty	0.05	0.17
library hold time	0.01	0.19
data required time		0.19

data required time		0.19
data arrival time		-5.21

slack (MET)		5.03

Typically, the STA output will include an error or warning indicating that the generated clock is not realizable. The best way to debug these kind of improper paths is to actually draw the clock waveforms at the capture flip-flop and at the launch flip-flop and try to understand if the edges being shown are indeed valid.

Lesson: Check the edges of the capture and launch clocks to see if they are indeed what they should be.

Missing Virtual Clock Latency

Consider the following path report.

```

Startpoint: RESET_L (input port clocked by VCLKM)
Endpoint: NPIWRAP/REG_25
(rising edge-triggered flip-flop clocked by CLKM)
Path Group: CLKM
Path Type: max

```

Point	Incr	Path
-----	-----	-----
clock VCLKM (rise edge)	0.00	0.00
clock network delay	0.00	0.00
input external delay	2.55	2.55 f
RESET_L (in) <-	0.00	2.55 f
. . .		
NPIWRAP/REG_25/D (DFF)	0.00	2.65 f
data arrival time		2.65
clock CLKM (rise edge)	10.00	10.00
. . .		

It is a path that starts from an input pin. Notice that the starting arrival time is listed as 0. This indicates that there was no latency specified on the clock *VCLKM* - the clock used to define the input arrival time on the input pin *RESET_L*; most probably this is a virtual clock, and that is why the arrival time is missing.

Lesson: When using virtual clocks, make sure latencies on virtual clocks are specified or are accounted for in the *set_input_delay* and *set_output_delay* constraints.

Large I/O Delays

When input or output paths have timing violations, the first thing to check is the latency on the clock used as reference to specify the input arrival time or the output required time. This is also applicable for the previous example.

The second thing to check is the input or output delays, that is, the input arrival time on an input path or the output required time on an output path. Quite often, one may find that these numbers are unrealistic for the target frequency. The input arrival time is usually the first value in the data path of the report, while the output required time is usually the last value in the data path of the report.

Point	Incr	Path
-----	-----	-----
clock VIRTUAL_CLKM (rise edge)	0.00	0.00
clock network delay	0.00	0.00
input external delay	14.00	14.00 f
PORT NIP (in) <-	0.00	14.00 f
UINV1/ZN (INV)	0.34	14.34 r
UAND0/Z (AN2)	0.61	14.95 r
UINV2/ZN (INV)	0.82	15.77 f
...		

In this data path of an input failing path, notice the input arrival time of 14ns. In this particular case, there was an error in the input arrival time specification in that it was too large.

Lesson: When reviewing input or output paths, check if the external delay specified is reasonable.

Incorrect I/O Buffer Delay

When a path goes through an input or an output buffer, it is possible for an incorrect specification to cause large delay values for the input or output buffer delays. In the case shown below, notice the large output buffer delay of 18ns; this is caused by a large load value specified on the output pin.

Startpoint: UFF4 (rising edge-triggered flip-flop clocked by CLKP)
 Endpoint: ROUT (output port clocked by VIRTUAL_CLKP)
 Path Group: VIRTUAL_CLKP
 Path Type: max

Point	Incr	Path
-----	-----	-----
clock CLKP (rise edge)	0.00	0.00
clock source latency	0.00	0.00
CLKP (in)	0.00	0.00 r
UCKBUF4/C (CKB)	0.06	0.06 r
UCKBUF5/C (CKB)	0.06	0.12 r

UFF4/CK (DFF)	0.00	0.12 r
UFF4/Q (DFF)	0.13	0.25 r
UBUF3/Z (BUFF)	0.09	0.33 r
IO_1/PAD:OUT (DDR11)	18.00	18.33
ROUT (out)	0.00	18.33 r
data arrival time		18.33
. . .		

Lesson: Watch out for large delays on buffers caused by incorrect load specifications.

Incorrect Latency Numbers

When a timing path fails, one thing to check is if the latencies of the launch clock and the capture clock are reasonable, that is, ensure that the skew between these clocks is within acceptable limits. Either an incorrect latency specification or incorrect clock balancing during clock construction can cause large skew in the launch and capture clock paths leading to timing violations.

Lesson: Check if clock skew is within reasonable limits.

Half-cycle Path

As mentioned in an earlier example, one needs to check the clock domains of the failing path. Along with this, one may need to check the edges at which the launch and capture flip-flops are being clocked. In some cases, one may find a half-cycle path - a rise to fall path or a fall to rise path - and it may be unrealistic to meet timing with a half-cycle path, or maybe the half-cycle path is not real.

Lesson: Ensure that data path has sufficient time to propagate.

Large Delays and Transition Times

One key item is to check for unusually large values for the delays or transition times along the data path. Some of these can be due to:

- *High-fanout nets*: Nets which are not buffered properly.
- *Long nets*: Nets which need buffer insertion in between.
- *Low strength cells*: Cells which may not have been replaced because these are labeled as don't touch in the design.
- *Memory paths*: Paths that typically fail due to large setup times on memory inputs and large output delays on memory outputs.

Missing Multicycle Hold

For a multicycle N setup specification, it is common to see the corresponding multicycle $N-1$ hold specification missing. Consequently, this can cause a large number of unnecessary delay cells to get inserted when a tool is fixing the hold violations.

Lesson: Always audit the hold violations *before* fixing to ensure that the hold violations that are being fixed are real.

Path Not Optimized

STA violations may be present on a path that has not been optimized yet. One can determine this situation by examining the data path. Are there cells with large delays? Can one manually improve the timing on the data path? Maybe the data path needs to be optimized more. It is possible that the tool is working on other worse violating paths.

Path Still Not Meeting Timing

If the data path appears to have good strong cells and if the path is still failing timing, one needs to examine the pins where the routing delay and wireload is high. This can be the next source of improvement. Maybe the cells can be moved closer and consequently the wireload and the wire routing delay can be decreased.

What if Timing Still Cannot be Met

One can utilize **useful skew** to help close the timing. Useful skew is where one purposely imbalances the clock trees, especially the launch and capture clock paths of a failing path so that the timing passes on that path. It typically means that the capture clock can be delayed so that the clock at the capture flip-flop arrives later when the data is ready. This does assume that there is enough slack on the succeeding data paths, that is, the data path for the next stage of flip-flop to flip-flop paths.

The reverse can also be attempted, that is, the launch clock path can be made shorter so that the data from the launch flip-flop is launched earlier to help meet the setup timing. Once again this can only be done if the preceding stage of flip-flop to flip-flop paths have the extra slack to give away.

Useful skew techniques can be used to fix both setup and hold violations. One disadvantage of this technique is that if the design has multiple modes of operation, then useful skew can potentially cause a problem in another mode.

10.11 Validating Timing Constraints

As chip size grows, there is more and more dependence on signing off timing with static timing analysis. The risk of relying only upon STA is that the STA is dependent on how good the timing constraints are. Therefore, validation of timing constraints becomes an important consideration.

Checking Path Exceptions

There are tools available that check the validity of false paths and multicycle paths based on the structure (netlist) of the design. These tools determine whether a given false path or multicycle path specification is valid. In addition, these tools may also be able to generate missing false path and multicycle path specifications based upon the structure of the design. However, some of the path exceptions generated by the tools may not be valid. This is because these tools determine the proof of a false path or a

multicycle path by the structure of the logic, typically using formal verification techniques, whereas a designer has a more in-depth knowledge of the functional behavior of the design. Thus the path exceptions generated by the tools need to be reviewed by the designer before accepting these and using them in STA. There may also be additional path exceptions that are based upon the semantic behavior of the design that have to be defined by the designer if the tool is unable to extract such exceptions.

The biggest risk in timing constraints are the path exceptions. Thus, false paths and multicycle paths should be determined after a careful analysis. In general, it is preferable to use a multicycle path as opposed to a false path. This ensures that the path in question is at least constrained by some amount. If a signal is sampled at a known or a predictable time, no matter how far out, use a multicycle path so that static timing analysis has some constraints to work with. False paths have the danger of causing timing optimization tools to completely ignore such paths, whereas in reality, they may indeed be getting sampled after some large number of clock cycles.

Checking Clock Domain Crossing

Tools are available to ensure that all clock domain crossings in a design are valid. These tools may also have the capability to automatically generate the necessary false path specifications. Such tools may also be able to identify illegal clock domain crossing, that is, cases where data is crossing two different clock domains without any clock synchronization logic. In such cases, the tools may provide the capability to automatically insert suitable clock synchronization logic where required. Note that not all asynchronous clock domain crossings require clock synchronizers. The requirement depends upon the nature of the data and whether it needs to be captured on the next cycle or a few cycles later.

An alternate way of checking asynchronous clock crossings using STA is to set a large clock uncertainty that is equal to the period of the sampling clock. This ensures that there will at least be some violations based upon which one can determine the appropriate path exceptions, or add the clock synchronization logic to the design.

Validating IO and Clock Constraints

Validating IO and clock constraints are still a challenge. Quite often timing simulations are performed to check the validity of all clocks in the design. System timing simulations are performed to validate the IO timing to ensure that the chip can communicate with its peripherals without any timing issues.

